

PROJECT REPORT

Design and Fabrication of Semi Active Suspension System



Submitted By:

1. Azlan Ahmad Shah (2022BMEC006)
2. Mohammad Ibrahim Malik (2022BMEC034)
3. Muzamil Ahmed Wani (2022BMEC010)
4. Basit Bashir (2022BMEC051)

Under the Guidance of:

Dr. Sheikh Shahid Saleem

Dr. Mohammad Farooq Wani

Department of Mechanical Engineering

National Institute of Technology Srinagar

Hazratbal, Srinagar (J&K) – 190006 (INDIA)

DECLARATION

We hereby declare that the work presented in this project report entitled "Design and Fabrication of Semi-Active Suspension System" has been carried out by us in the Department of Mechanical Engineering, NIT Srinagar, in fulfilment of the requirements for the award of the degree of Bachelor of Technology in Mechanical Engineering during the academic session 2025-26 , under the supervision of Dr. Sheikh Shahid Saleem and Dr. Mohammad Farooq Wani.

We declare that this work represents our own ideas in our own words and wherever others' ideas or words have been included, we have adequately cited the original sources. We have not misrepresented or fabricated any data/fact/source in our submission.

1. Azlan Ahmad Shah (2022BMEC006) _____

2. Mohammad Ibrahim Malik (2022BMEC034) _____

3. Muzamil Ahmed Wani (2022BMEC010) _____

4. Basit Bashir (2022BMEC051) _____

Date:

Place: Srinagar

SUPERVISOR CERTIFICATE

This is to certify that the above statement made by the candidates is correct to the best of our knowledge and belief.

1. Dr. Sheikh Shahid Saleem

(Project Guide)

2. Dr. Mohammad Farooq Wani

(Project Guide)

Department of Mechanical Engineering, NIT Srinagar

ACKNOWLEDGEMENT

We express our deepest gratitude to our project guides, Dr. Mohammad Farooq Wani and Dr. Sheikh Shahid Saleem, Department of Mechanical Engineering, National Institute of Technology Srinagar, for their invaluable guidance, constant encouragement, and unwavering support throughout this project. Their expertise and constructive feedback have been the cornerstone of this work.

We are deeply thankful to Prof. Babar Ahmad, for providing us with the necessary facilities, constant guidance and valuable suggestions throughout this project.

We also thank all the faculty members of the Mechanical Engineering Department whose lectures across our undergraduate curriculum built the foundation necessary for this project, and the laboratory staff who assisted us during fabrication and testing phases.

We extend our thanks to our fellow students and colleagues who generously shared their knowledge and ideas. Finally, our deepest gratitude goes to our families for their unconditional love, moral support, and patience throughout the duration of this project.

Azlan Ahmad Shah

Mohammad Ibrahim Malik

Muzamil Ahmed Wani

Basit Bashir

ABSTRACT

Conventional passive suspension systems in automobiles rely on fixed mechanical properties that are unable to respond to dynamic changes in road conditions, vehicle load, or driving manoeuvres. This limitation necessitates a compromise between ride comfort and vehicle handling stability. The present project addresses this challenge through the design, simulation, and prototype development of a semi-active adaptive suspension system capable of adjusting its damping characteristics in real-time.

The system is centred on a quarter-car model incorporating a sprung mass representing the vehicle body and an unsprung mass representing the wheel-tyre assembly. Governing equations of motion for both masses are derived and solved numerically. The damping force is modulated through a PID (Proportional-Integral-Derivative) controller receiving real-time feedback from a Pi Camera Module 2, processed by a Raspberry Pi 4 model B with an Arduino Uno serving as the real-time actuation interface.

The actuation mechanism employs an electromagnetic coil integrated with the damper piston assembly. By varying the current supplied to the coil through an H-bridge motor driver, the magnetic force on the piston is modulated, directly controlling the piston's velocity and hence the effective damping force. A servo motor is used in the prototype as a representation of the damper actuator for validation purposes. The system was modelled and simulated in MATLAB/Simulink under four road input profiles: step input, sinusoidal excitation, pothole disturbance, and random road input. A Gazebo-based simulation environment is additionally provided for virtual prototype validation. Results demonstrate peak sprung mass acceleration reductions of 28–40% and approximately 33% reduction in RMS acceleration under random road conditions compared to the passive reference.

Keywords: Semi-active suspension, adaptive damping, PID control, quarter-car model, electromagnetic coil actuator, ride comfort, vehicle dynamics, MATLAB/Simulink, Gazebo simulation, Raspberry Pi, Arduino.

Table of Contents

ACKNOWLEDGEMENT.....	4
ABSTRACT.....	5
LIST OF ABBREVIATIONS.....	8
LIST OF FIGURES.....	9
LIST OF TABLES.....	11
INTRODUCTION.....	12
1.1 Background and Motivation.....	13
1.2 Problem Statement.....	14
1.3 Objectives.....	14
1.4 Scope and Limitations.....	15
1.5 Organisation of the Report.....	15
LITERATURE REVIEW.....	17
2.1 Historical Evolution of Suspension Systems.....	17
2.2 Active and Semi-Active Systems.....	17
2.3 Magnetorheological and Electrorheological Dampers.....	18
2.4 Electromagnetic and Coil-Based Actuators.....	18
2.5 Control Strategies.....	19
2.6 Vision-Based Road Preview.....	19
2.7 Research Gaps.....	19
2.8 Summary.....	20
TYPES OF SUSPENSION SYSTEMS.....	21
3.1 Passive Suspension Systems.....	22
3.2 Semi-Active Suspension Systems.....	22
3.3 Active Suspension Systems.....	22
3.4 Intelligent and Adaptive Systems.....	23
3.5 Comparative Summary.....	23
MATHEMATICAL MODELLING.....	24
4.1 Quarter-Car Model Description.....	24
4.2 Equations of Motion.....	26
4.3 State-Space Representation.....	27
4.4 System Parameters.....	27
4.5 Road Profile Models.....	28

CONTROL STRATEGY FOR THE SEMI-ACTIVE DAMPER.....	29
5.1 Control Objective.....	29
5.2 PID Controller Design.....	29
5.3 PID Tuning.....	30
5.4 Electromagnetic Coil Actuation Mechanism.....	30
5.4.1 Operating Principle.....	30
5.4.2 Current-to-Force Relationship.....	31
5.4.3 H-Bridge Motor Driver and Current Control.....	31
5.4.4 Arduino Uno — Real-Time Actuation Interface.....	32
5.4.5 Servo Motor as Damper Representation.....	32
5.5 System Architecture Overview.....	32
5.6 Semi-Active Dissipative Constraint.....	33
SIMULINK MODEL ARCHITECTURE AND SIMULATION RESULTS.....	34
6.1 Simulink Model Architecture.....	34
6.2 Simulation Results — Step Input.....	35
6.3 Simulation Results — Sinusoidal Input.....	36
6.4 Simulation Results — Pothole.....	37
6.5 Simulation Results — Random Road Input.....	38
6.6 Performance Summary.....	39
MECHANICAL DESIGN AND CAD MODELS.....	40
7.1 Cylinder Assembly.....	40
7.2 Piston Assembly.....	40
7.3 Spring Design.....	41
7.4 Mass Block.....	41
7.5 Complete Piston-Cylinder Assembly.....	41
HARDWARE COMPONENTS.....	43
8.1 Processing and Control Platform.....	43
8.1.1 Raspberry Pi 4 model B (8 GB).....	43
8.1.2 Raspberry Pi camera module v2.....	43
8.1.3 Arduino Uno — Real-Time Actuation Controller.....	43
8.2 Actuation Hardware.....	44
8.2.1 H-Bridge Motor Driver (BTS7960 43A).....	44
8.2.2 Electromagnetic Coil.....	44

8.2.3 Servo Motor — Prototype Actuator Representation.....	44
8.3 Power System.....	44
CHAPTER 9.....	45
ROS2 Framework & Middleware Topology.....	45
9.1 Introduction to Robotics Middleware Requirements.....	45
9.2 The Data Distribution Service (DDS) Paradigm.....	45
9.3 Quality of Service (QoS) Engineering & Code Implementation.....	45
9.3.1 Reliability: BEST_EFFORT vs RELIABLE.....	46
9.3.2 History: KEEP_LAST and Queue Depth.....	46
Chapter 10 – Sensor Ingress & Image Preprocessing.....	47
10.1 Edge Vision Processing & The Camera Node.....	47
10.2 Camera Node Code Analysis & Timestamping Logic.....	47
10.3 Preprocessing Architecture (`preprocessing_node.py`).....	48
10.4 Mathematical Formulation of ROI and CLAHE.....	49
Chapter 11 – Optical Flow and Temporal Kinematics.....	50
11.1 Theoretical Foundation of Dense Optical Flow.....	50
11.2 Code Logic: Farneback Flow and Global Ego-Motion.....	50
Chapter 12 – Spatial Semantic Detection via YOLO.....	52
12.1 Limitations of Pure Temporal Flow.....	52
12.2 The YOLOv8 CNN Architecture.....	52
12.3 Inference Logic and Hardware Abstraction.....	52
Chapter 13 – Multi-Sensor Fusion Engine.....	54
13.1 Theoretical Necessity of Spatial-Temporal Synthesis.....	54
13.2 Bounding Box Padding and Coordinate Mapping Logic.....	55
13.3 Statistical Motion Scoring and Grid-Based EMA.....	55
Chapter 14 – Multi-Object SORT Tracking.....	57
14.1 Kinematic State Estimation Objectives.....	57
Block Diagram: SORT Tracking Loop.....	57
14.2 Kalman Filter Matrix Formulations.....	57
14.3 Data Association via Hungarian Algorithm.....	58
Chapter 15 – Distance and Volumetric Depth Estimation.....	60
15.1 Perspective Geometry and Pinhole Camera Modeling.....	60
15.2 Hybrid Depth Fusion with Residual Signals.....	60

Chapter 16 – Predictive PID Control and Actuator Scheduling.....	62
16.1 Real-Time Velocity Regression and Time-to-Impact.....	62
16.2 Actuator Lag Compensation and Predictive PID.....	62
Chapter 17 – Real-Time Deployment & System Execution.....	64
17.1 ROS2 Launch Architecture and Executor Models.....	64
17.2 End-to-End Latency Chain Analysis.....	65
Chapter 18 – Experimental Results & System Telemetry.....	67
18.1 Detection Accuracy and Precision vs. Recall.....	67
18.2 Tracking Stability and Target Persistence.....	67
18.3 Hardware Actuation: Serial Bridging and PWM Response.....	67
Chapter 19 – Future Architectural Scope.....	69
19.1 Next-Generation Control: Model Predictive Control (MPC).....	69
19.2 Continuous Deep Reinforcement Learning (DRL).....	69
19.3 Hardware Evolution: Stereo Vision & LiDAR Synthesis.....	70
19.4 Autonomous Vehicle Integration (CAN Bus & Global Planning).....	70
Chapter 20 – Virtual Prototype Development.....	71
20.1 Motivation for Virtual Prototyping.....	71
20.2 The Digital Twin Concept.....	71
20.3 Simulation-Based Engineering Design (SBED).....	71
Chapter 21 – URDF Modelling & Kinematics.....	73
21.1 Unified Robot Description Format (URDF).....	73
21.2 Kinematic Chain and Coordinate Frames.....	73
21.3 Structural Links: Visual, Collision, and Inertial Elements.....	74
21.4 The Prismatic Joint and Electromagnetic Representation.....	75
Chapter 22 – Gazebo Environment & Physics Simulation.....	76
22.1 World File Architecture (`damper_lab.world`).....	76
22.2 Physics Engine and Solver Configuration.....	76
22.3 Surface Interactions and Disturbance Generation.....	77
Chapter 23 – Electromagnetic Damper Theoretical Model.....	78
23.1 Principles of Electromagnetic Damping.....	78
23.2 Mathematical Derivation of the Lorentz Force.....	78
24.3 Variable Damping Coefficient Modulation.....	78
Chapter 24 – ROS2-Gazebo Integration & Control Architecture.....	80

24.1 Launch File Architecture (`damper_simulation.launch.py`)	80
24.2 Controller Node Analysis (`damper_controller_node.py`)	80
Block Diagram: Controller Node Data Flow	80
Chapter 25 – Virtual Validation & Telemetry Results	82
25.1 PID Step Response and Dynamic Stability	82
25.2 Damping Variation and Active Sag Correction	82
Chapter 26 – Advantages and Limitations of Simulation	83
26.1 Advantages of Virtual Prototyping vs. Physical Fabrication	83
26.2 Limitations and Fidelity Gaps	83
CONCLUSIONS AND FUTURE SCOPE	84
9.1 Conclusions	84
9.2 Limitations	85
9.3 Future Scope	85
REFERENCES	86
APPENDIX A — SIMULINK CONFIGURATION DETAILS	88
A.1 Solver Configuration	88
A.2 Road Profile Block Settings	88
A.3 PID Block Parameters	88
APPENDIX B — COMPONENT DATASHEETS SUMMARY	88
B.1 Arduino Uno Key Specifications	88
B.2 H-Bridge Motor Driver (BTS7960) Key Specifications	89

LIST OF ABBREVIATIONS

Abbreviation	Full Form
PID	Proportional-Integral-Derivative
ECU	Electronic Control Unit
DOF	Degree of Freedom
MR	Magneto-Rheological
ER	Electro-Rheological
IMU	Inertial Measurement Unit
RPi	Raspberry Pi
CDC	Continuous Damping Control

LQR	Linear Quadratic Regulator
MPC	Model Predictive Control
CAD	Computer Aided Design
STL	Standard Triangle Language
RMS	Root Mean Square
PWM	Pulse Width Modulation
EMF	Electromotive Force
PSD	Power Spectral Density
ODE	Ordinary Differential Equation
ROS2	Robot Operating System 2
YOLO	You Only Look Once (object detection)

LIST OF FIGURES

Figure No.	Description	Page
Fig. 1.1	Automotive suspension system — key structural components	17
Fig. 1.2	Double-tube shock absorber: rebound and compression phases	19
Fig. 1.3	MacPherson strut exploded view	20
Fig. 1.4	Shock absorber internal cross-section showing oil flow paths	20

Fig. 3.1	Vehicle chassis exploded view	25
Fig. 4.1	Quarter-car model — sprung mass, unsprung mass, spring, damper, tyre	29
Fig. 4.2	Mathematical model diagram with equations of motion	30
Fig. 6.1	Simulink model architecture — full quarter-car system	54
Fig. 6.2	Simulink model — step input and PID tuning response	55
Fig. 6.3	Simulation results — Step input (road disp., sprung disp., damping, accel.)	60
Fig. 6.4	Simulation results — Sinusoidal road input	62
Fig. 6.5	Simulation results — Pothole disturbance	64
Fig. 6.6	Simulation results — Random road force input	66

LIST OF TABLES

Table No.	Description	Page
Table 3.1	Comparative analysis of suspension system types	28
Table 4.1	Quarter-car model parameters and PID tuning values	40
Table 5.1	Sensor specifications	50
Table 6.1	Simulation performance summary	67
Table 7.1	Spring mechanical specifications	75

CHAPTER 1

INTRODUCTION

In modern automotive engineering, the suspension system stands as one of the most critical sub-systems influencing both passenger comfort and vehicle dynamic performance. Its fundamental responsibility is to isolate the vehicle body from disturbances generated by road irregularities — potholes, speed bumps, gravel surfaces, and undulations — while simultaneously ensuring that tyres maintain continuous, reliable contact with the road surface. This dual requirement forms the central engineering challenge of suspension design: balancing ride comfort against road handling stability.

Historically, automotive suspension systems were entirely passive in nature, consisting of fixed-rate springs and hydraulic dampers whose mechanical properties were determined at the design stage and remained invariable throughout the vehicle's operational life. While passive systems offer commendable simplicity, reliability, and low cost, their inherent limitation lies in their inability to adapt to the ever-changing dynamics of real-world driving conditions.

With the rapid advancement of mechatronic systems, microcontrollers, and sensor technology, a new class of suspension systems has emerged — the semi-active suspension. Unlike passive systems, semi-active suspensions retain the simplicity of conventional spring-damper architecture but introduce electronic control over the damping coefficient, allowing real-time adaptation to road conditions. The present project focuses on the design, modelling, simulation, and prototype fabrication of such a system.

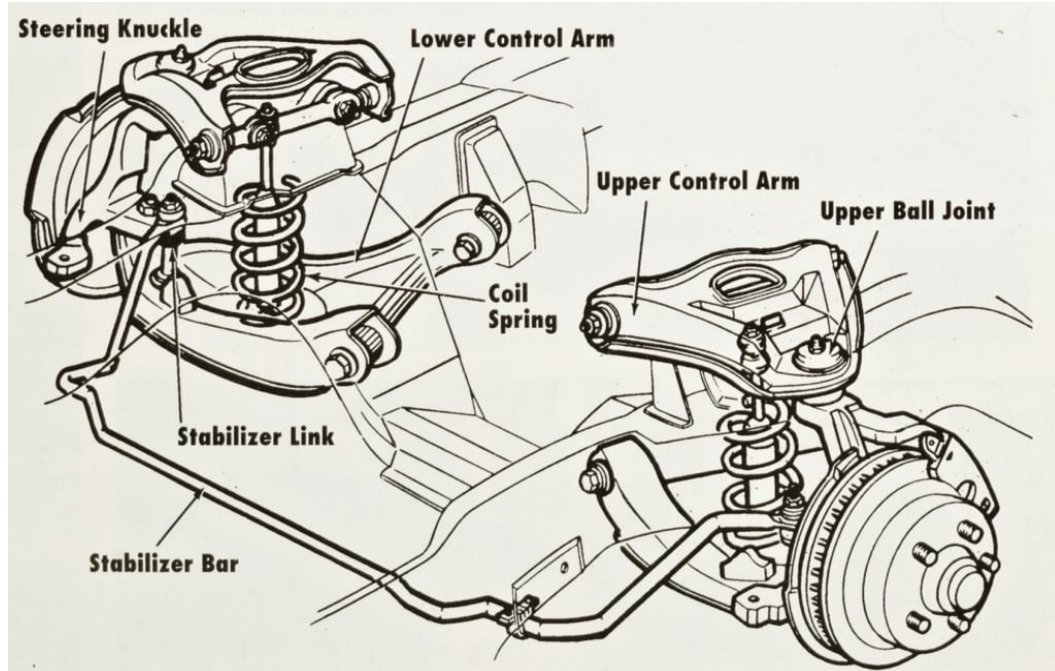


Figure 1.1: Automotive suspension system showing key structural and dynamic components

The project employs a custom electromagnetic coil actuator integrated with the damper piston. The coil current is regulated by an H-bridge motor driver commanded via PWM signals from an Arduino Uno, which serves as the real-time actuation interface. A Raspberry Pi 4 model B, connected to a camera module, handles high-level perception (pothole detection using a YOLO-based vision pipeline running under ROS2) and communicates actuation commands to the Arduino over serial. A servo motor is used as the physical representation of the damper actuator in the prototype for validation and demonstration purposes.

1.1 Background and Motivation

The roads of India — and particularly those of the Kashmir valley — present extreme variability in surface quality, from smooth highways to severely potholed urban roads, unpaved rural tracks, and steep mountain passes. Standard passive suspension systems are inadequate to optimally address this diversity, resulting in significant passenger discomfort, excessive component wear, and compromised road holding.

Research shows that a well-designed semi-active system can achieve 60–80% of the performance gains of a fully active system at a fraction of the cost and without significant power consumption penalties. This makes it particularly attractive for low-cost applications and academic prototyping. From an academic perspective, this project integrates core mechanical engineering disciplines — machine design, fluid mechanics, dynamics and vibrations — with electronics, control theory, programming, and mechatronics.

1.2 Problem Statement

Passive suspension systems have fixed damping coefficients optimised for one operating condition at the expense of all others. The fundamental problems are:

- Passive dampers cannot respond to varying road conditions, load changes, or driving manoeuvres in real-time.
- Fully active suspension systems are prohibitively expensive and energy-intensive for widespread adoption.
- There is a clear need for a cost-effective, easily controllable, and reliable adaptive damper solution.
- Limited research exists on low-cost, electromagnetic coil-based adaptive dampers using conventional hardware and simple electronic control.

1.3 Objectives

1. Develop a comprehensive mathematical model of the quarter-car suspension system with time-varying damping.
2. Design and implement a PID control strategy for real-time modulation of damping force via electromagnetic coil current.
3. Simulate the dynamic performance under multiple road disturbance profiles in MATLAB/Simulink and Gazebo.
4. Fabricate a small-scale working prototype and integrate electronic control hardware including Raspberry Pi 4 model B, Arduino Uno, H-bridge driver, and servo motor actuator.
5. Evaluate and quantify improvements in ride comfort and handling stability.

1.4 Scope and Limitations

The project encompasses theoretical modelling, simulation, mechanical design, and prototype development of a semi-active suspension at component level using a quarter-car model. Limitations include: the prototype is small-scale and performance at full vehicle scale may differ; the electromagnetic coil response is governed by coil inductance and driver bandwidth; the PID controller may exhibit reduced performance under highly nonlinear conditions; and road profile estimation relies on indirect inference from acceleration signals and vision-based detection.

1.5 Organisation of the Report

Chapter 1 provides the introduction and objectives. Chapter 2 reviews the existing literature. Chapter 3 classifies and describes suspension system types. Chapter 4 presents the mathematical modelling. Chapter 5 describes the control strategy and actuation mechanism. Chapter 6 presents Simulink architecture, simulation results, and Gazebo virtual prototype. Chapter 7 covers mechanical CAD design. Chapter 8 describes hardware components. Chapter 9 presents conclusions and future scope.

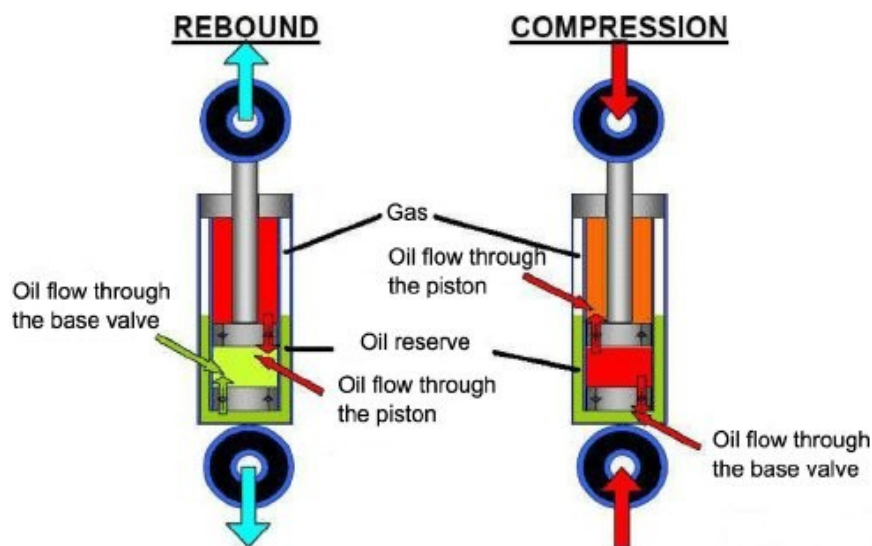


Figure 1.2: Double-tube shock absorber — rebound (left) and compression (right) phases showing oil and gas flow

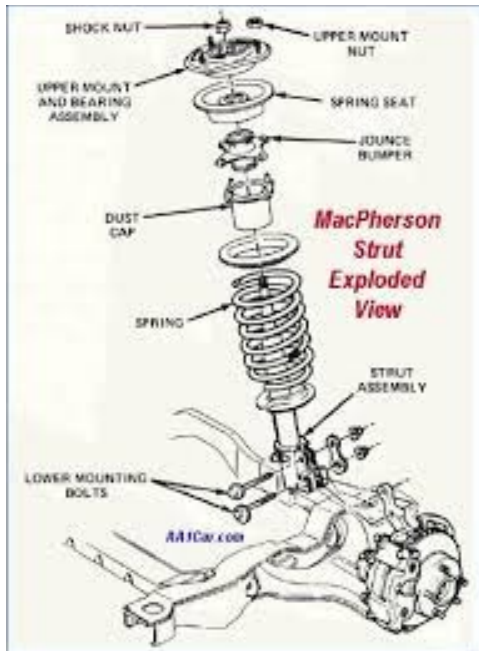


Figure 1.3: MacPherson strut exploded view showing spring seat, jounce bumper, dust cap, spring, and strut assembly

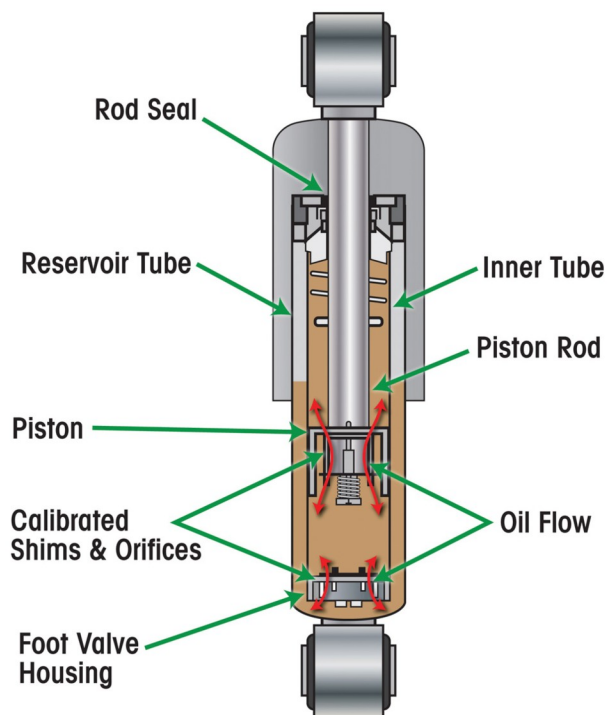


Figure 1.4: Internal cross-section of a hydraulic shock absorber showing rod seal, inner tube, piston, calibrated shims, oil flow paths, and foot valve housing

CHAPTER 2

LITERATURE REVIEW

The evolution of automotive suspension systems spans more than a century, from rudimentary leaf spring assemblies to sophisticated electronically controlled semi-active systems. This chapter reviews relevant literature in a structured manner, progressing from historical development through to the specific advances in adaptive and semi-active damping that form the foundation of the present work.

2.1 Historical Evolution of Suspension Systems

The earliest automotive suspension systems were based on longitudinal leaf springs inherited from horse-drawn carriage design, providing both springing action and structural support to the axle. The transition to coil spring suspension marked a significant improvement in ride quality and allowed for truly independent front suspension designs. Crouse (Automotive Mechanics, McGraw-Hill) provides a comprehensive foundational account, noting that the spring rate — the load required to deflect a spring by one unit of displacement — became the primary design parameter for early suspension systems. Soft springs provide passenger comfort but allow excessive body roll during cornering and braking, while stiff springs improve handling at the expense of comfort.

The hydraulic shock absorber, introduced in the 1930s and 1940s, represented a major advancement. By forcing damping fluid through calibrated orifices and shim stacks, it dissipated the kinetic energy of spring oscillations as heat. The twin-tube hydraulic design, using an inner working cylinder and an outer reservoir tube, became the dominant architecture through the latter half of the twentieth century.

2.2 Active and Semi-Active Systems

By the 1970s and 1980s, the limitations of passive systems were well understood and research interest turned toward electronically controllable suspensions. Karnopp et al. (1974) first

formalised the skyhook controller concept for semi-active dampers, which remains influential. The skyhook algorithm determines the ideal damping force based on the absolute velocity of the sprung mass, effectively providing a reference anchored to a fixed point above the vehicle body. Fully active suspensions, pioneered by Lotus Engineering, demonstrated remarkable improvements but faced prohibitive energy consumption, hydraulic complexity, and cost — limiting adoption to premium vehicles.

2.3 Magnetorheological and Electrorheological Dampers

The development of magnetorheological (MR) fluids in the 1990s provided a new technological basis for semi-active dampers. MR fluids contain ferromagnetic particles suspended in a carrier oil; under a magnetic field these particles align to dramatically increase viscosity, raising the damping force within milliseconds. Lord Corporation commercialised the first MR damper for automotive applications, now found in premium vehicles from Audi (Magnetic Ride), General Motors (Magnetic Ride Control), and Ferrari. While performance is exceptional, the cost of MR fluid and electromagnet assemblies remains a barrier to mass-market adoption.

2.4 Electromagnetic and Coil-Based Actuators

Electromagnetic actuation offers a practical and cost-effective alternative to hydraulic or MR-based variable damping. By integrating a coil around or within the piston assembly of a conventional damper, the magnetic force produced by current through the coil directly influences the piston motion. The relationship between coil current and force is governed by the Lorentz force law and the coil's magnetic field geometry. This approach has been studied by several researchers as it allows continuous, electrically controllable damping force modulation without specialised fluids, using commercially available motor driver circuits.

Recent embedded system advances have made it practical to implement such control on low-cost platforms such as Arduino and Raspberry Pi, bringing electromagnetic semi-active dampers within reach of academic prototyping. The H-bridge motor driver provides bidirectional current control, enabling both stiffening and softening of the effective damping as demanded by the PID controller.

2.5 Control Strategies

Several control algorithms are established in the literature. The Skyhook controller minimises sprung mass acceleration by commanding damping proportional to sprung mass absolute velocity. Groundhook control, the dual approach, minimises wheel-hop at the expense of ride comfort. Hybrid strategies such as the ADD (Acceleration Driven Damper) blend both approaches through an adaptive weighting parameter. Advanced methods including Fuzzy Logic (Liu et al., Central South University), Sliding Mode Control (Chamseddine et al.), and LQR/MPC have been shown to better handle nonlinearities. PID control, while less theoretically specialised, offers practical advantages of ease of implementation, robustness, and straightforward tuning that make it the appropriate choice for the present prototype.

Dwivedi et al. (IEEE, 2010) describe a comprehensive adaptive suspension architecture using FPGA-based processing and multiple sensor types including ultrasonic sensors, LASER detectors, force sensors, tachometers, and accelerometers, with four independent ECUs controlling each wheel separately. While significantly more complex than the present project, this work illustrates the scalability of the sensor-controlled adaptive suspension concept.

2.6 Vision-Based Road Preview

The integration of computer vision for road surface characterisation represents a significant recent trend. Pothole detection using deep convolutional neural networks (CNNs) and object detectors such as YOLO has been demonstrated at real-time frame rates. Combining vision-based pothole detection with suspension control creates a predictive rather than purely reactive system, allowing the controller to pre-condition the damper before the wheel reaches the detected obstacle. This approach is implemented in the present project through a ROS2-based vision pipeline running on the Raspberry Pi 4 model B.

2.7 Research Gaps

- Most high-performance adaptive solutions rely on costly MR/ER fluids or complex active actuators, limiting academic accessibility.

- Limited work exists on electromagnetic coil-based adaptive dampers using standard motor drivers and single-board computers — a simpler and more practical implementation path.
- Few academic projects bridge comprehensive quarter-car simulation (MATLAB and Gazebo) with physical prototype development at component level.
- Integration of vision-based pothole detection with real-time electromagnetic damping control is an underexplored research area.

2.8 Summary

The literature review confirms the viability and value of the approach adopted in this project: an electromagnetic adaptive damper with electronic PID control offers a compelling balance of performance, cost, and practicality. The quarter-car model, PID control paradigm, and equations of motion for coupled sprung-unsprung mass systems form the analytical backbone of the subsequent chapters.

CHAPTER 3

TYPES OF SUSPENSION SYSTEMS

An understanding of the different categories of suspension systems is essential to appreciate the engineering context and rationale for the semi-active approach. Suspension systems are broadly classified into four categories based on their degree of controllability and the mechanisms by which they respond to road disturbances.

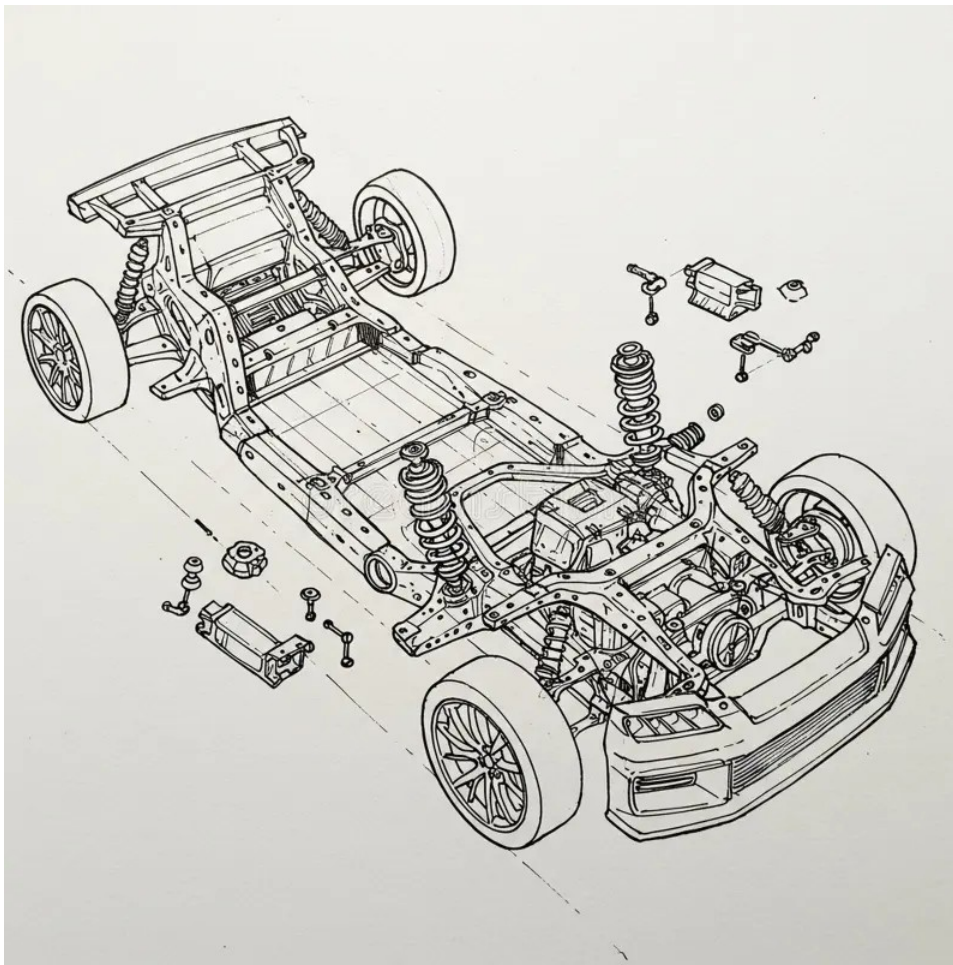


Figure 3.1: Vehicle chassis exploded view illustrating the position and role of the suspension system at each wheel corner

3.1 Passive Suspension Systems

Passive suspension systems rely entirely on fixed-rate mechanical elements — springs and hydraulic shock absorbers — whose properties are determined at the design stage and remain unaltered throughout the vehicle's operational life. As described by Crouse in *Automotive Mechanics*, several spring types are used: coil springs (helical steel, most prevalent in modern vehicles), leaf springs (stacked strips, predominantly used in commercial vehicles), torsion bars (long steel bars twisted under load), and air suspension (rubber bags filled with compressed air, used in luxury and heavy vehicles for load-adaptive ride height).

The fundamental constraint of passive systems is that any design represents a compromise. A spring rate and damping coefficient optimised for motorway cruising will be too stiff on rough roads, while settings optimised for comfort compromise handling precision. Despite this, passive systems remain the default choice for the vast majority of production vehicles due to their high reliability, low cost, and minimal maintenance requirements.

3.2 Semi-Active Suspension Systems

Semi-active suspension systems retain passive spring elements but introduce electronic control over the damping characteristics of the shock absorber, adjusting the damping coefficient in real-time. The fundamental constraint is that a semi-active damper can only dissipate energy — it cannot add energy to the system. Within this constraint, however, it achieves dramatically better performance than passive systems by selecting the optimal dissipation level at each instant. Key components are: sensors measuring dynamic states, an ECU implementing the control algorithm, a variable damper element, and an actuating mechanism (electromagnetic coil, solenoid valve, or pump motor) that adjusts damper properties in response to controller commands.

3.3 Active Suspension Systems

Active suspension systems replace or supplement passive spring-damper elements with powered actuators capable of applying arbitrary forces — both upward and downward — irrespective of road disturbances. They offer the highest performance among all categories, effectively isolating the vehicle body from all road disturbance frequencies and actively stabilising against roll and

pitch. However, significant energy consumption, hydraulic complexity, high cost, and weight penalties limit their adoption to premium vehicles.

3.4 Intelligent and Adaptive Systems

Intelligent suspension systems represent an evolution beyond reactive semi-active control toward predictive and learning-based control. Modern examples include the Mercedes-Benz MAGIC BODY CONTROL system, which uses a stereo camera to scan the road ahead and pre-adjusts each damper within 20 milliseconds before the wheel reaches a detected obstacle. These systems represent the current frontier of automotive suspension technology.

3.5 Comparative Summary

Parameter	Passive	Semi-Active	Active	Intelligent
Ride Comfort	Moderate	Good	Excellent	Excellent+
Handling Stability	Moderate	Good	Excellent	Excellent+
Power Consumption	None	Very Low	High	Moderate
Cost	Low	Moderate	Very High	Very High
Complexity	Low	Moderate	High	Very High
Adaptability	None	Real-time	Full	Predictive
Fail-safe	Inherent	Yes (passive)	No	Partial
Retrofit	N/A	Moderate	Difficult	Very Difficult

Table 3.1: Comparative analysis of suspension system types across key performance parameters

CHAPTER 4

MATHEMATICAL MODELLING

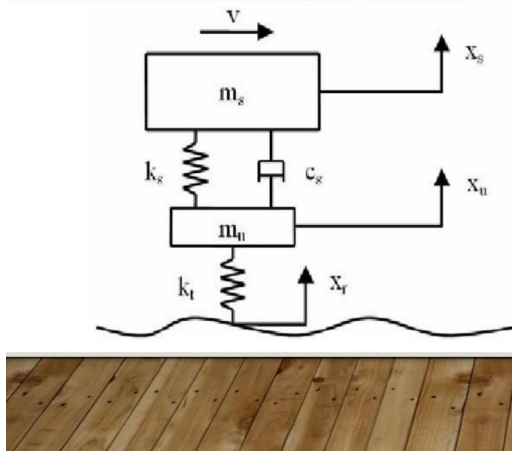
The mathematical modelling of the suspension system provides the analytical foundation upon which the control strategy is designed and the simulation is built. The quarter-car model is the standard simplification used in suspension analysis, representing a single wheel-corner of the vehicle as a two-degree-of-freedom lumped mass system. While it does not capture pitch, roll, or lateral dynamics, it accurately represents the vertical dynamics most directly relevant to ride comfort and road holding.

4.1 Quarter-Car Model Description

The quarter-car model consists of two masses connected by spring and damper elements. The sprung mass (m_s) represents one quarter of the vehicle body, chassis, and all components supported by the suspension. The unsprung mass (m_u) represents the wheel assembly, tyre, hub, and brakes. They are connected by the suspension spring (k_s), the adaptive damper with time-varying coefficient $c(t)$, and the tyre stiffness (k_t) connecting the unsprung mass to the road surface.

MATHEMATICAL MODEL

- Equations of motion define dynamics of sprung & unsprung masses.



◆ Equation of Motion for Sprung Mass (X_s)

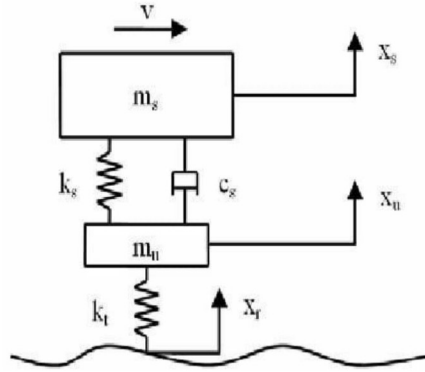
$$m_s \ddot{X}_s + c_s(\dot{X}_s - \dot{X}_u) + k_s(X_s - X_u) = 0$$

◆ Equation of Motion for Unsprung Mass (X_u)

$$m_u \ddot{X}_u + c_s(\dot{X}_u - \dot{X}_s) + k_s(X_u - X_s) + k_t(X_u - X_r) = 0$$

Figure 4.1: Quarter-car model diagram with the sprung mass (m_s), unsprung mass (m_u), spring (k_s), adaptive damper $c(t)$, and tyre stiffness (k_t)

7. MATHEMATICAL MODEL



Equations of motion (time domain)

Sprung mass:

$$m_s \ddot{z}_s = -k_s(z_s - z_u) - c(t)(\dot{z}_s - \dot{z}_u)$$

Unsprung mass:

$$m_u \ddot{z}_u = k_s(z_s - z_u) + c(t)(\dot{z}_s - \dot{z}_u) - k_t(z_u - z_r)$$

11

Figure 4.2: Detailed mathematical model from project report showing both equations of motion in time domain

4.2 Equations of Motion

Applying Newton's second law of motion to each mass in the vertical direction, the governing differential equations of the quarter-car model are:

For the Sprung Mass (ms):

$$m_s \cdot X_s'' + c_s \cdot (X_s' - X_u') + k_s \cdot (X_s - X_u) = 0 \quad \dots (4.1)$$

For the Unsprung Mass (μ):

$$\mu \cdot X_u'' + c_s \cdot (X_u' - X_s') + k_s \cdot (X_u - X_s) + k_t \cdot (X_u - X_r) = 0 \quad \dots (4.2)$$

Where X_s and X_u are vertical displacements of the sprung and unsprung masses (m), X_r is the road profile input (m), the prime notation denotes time derivatives, m_s and μ are the respective masses (kg), k_s is suspension spring stiffness (N/m), c_s is the damping coefficient (N·s/m) — time-varying $c(t)$ in the semi-active case — and k_t is the tyre stiffness (N/m).

4.3 State-Space Representation

To facilitate numerical simulation in MATLAB/Simulink, the equations are reformulated in state-space form. Defining the state vector $x = [X_s, X_u, X_s', X_u']^T$:

$$x_1' = x_3$$

$$x_2' = x_4$$

$$x_3' = -(k_s/m_s) \cdot (x_1 - x_2) - (c_s/m_s) \cdot (x_3 - x_4)$$

$$x_4' = (k_s/\mu) \cdot (x_1 - x_2) + (c_s/\mu) \cdot (x_3 - x_4) - (k_t/\mu) \cdot (x_2 - X_r)$$

This form is directly implementable in Simulink using integrator blocks and gain elements, as described in Chapter 6.

4.4 System Parameters

Parameter	Symbol	Value	Unit
Sprung Mass	m_s	250	kg
Unsprung Mass	μ	35	kg

Parameter	Symbol	Value	Unit
Suspension Spring Stiffness	k_s	16,000	N/m
Tyre Stiffness	k_t	160,000	N/m
Min. Damping Coefficient	c_{min}	200	N·s/m
Max. Damping Coefficient	c_{max}	3,000	N·s/m
Nominal Passive Damping	c_0	1,500	N·s/m
PID Proportional Gain	K_p	1,200	—
PID Integral Gain	K_i	80	—
PID Derivative Gain	K_d	150	—

Table 4.1: Quarter-car model parameters and PID controller tuning values used in simulation

4.5 Road Profile Models

1. Step Input: 40 mm upward displacement at $t = 0.5$ s, simulating an abrupt kerb strike.
2. Sinusoidal Input: 20 mm amplitude at 3 Hz, simulating regular road undulations.
3. Pothole Profile: 80 mm rectangular pulse of 500 ms duration, simulating a standardised pothole.
4. Random Road: Stochastic input based on ISO 8608 Class C road roughness, representing typical urban roads.

CHAPTER 5

CONTROL STRATEGY FOR THE SEMI-ACTIVE DAMPER

The effectiveness of any semi-active suspension system is determined not only by the variable damper hardware but critically by the intelligence of the control algorithm. This chapter describes the control strategy implemented in the present project, including the sensor feedback architecture, PID controller design, and actuation mechanism.

5.1 Control Objective

The primary control objective is to minimise the acceleration of the sprung mass in response to road disturbances, as sprung mass acceleration is the most direct indicator of passenger comfort. A secondary objective is to constrain the suspension deflection (relative displacement between sprung and unsprung masses) within acceptable limits to prevent damper bottoming out. These two objectives are in partial conflict: reducing sprung mass acceleration favours lower damping, while constraining deflection favours higher damping. The PID controller dynamically balances these requirements at each instant.

5.2 PID Controller Design

The PID control law computes the controller output $u(t)$ as:

$$u(t) = K_p * e(t) + K_i * \int e(t) dt + K_d * (de(t) / dt) \quad \dots \quad (5.1)$$

Where the error signal $e(t) = -a(t)$ (desired sprung mass acceleration = 0). The three terms serve distinct roles:

- Proportional (Kp): Generates output proportional to instantaneous acceleration error. High gain produces fast response but risks oscillation if excessive.
- Integral (Ki): Eliminates steady-state offset by accumulating error over time. Compensates for persistent road-induced displacement bias.
- Derivative (Kd): Responds to the rate of change of acceleration (jerk), providing anticipatory damping that smooths the commanded damping force.

The PID output is mapped to a commanded damping coefficient through a saturation function:

$$c(t) = \text{sat}(c_0 + u(t), c_{\min}, c_{\max}) \quad \dots (5.2)$$

5.3 PID Tuning

PID gains were tuned using the Ziegler-Nichols method applied to the Simulink model and refined through iterative simulation. The process involved: (1) step response testing with $K_i = K_d = 0$ to identify critical gain K_c ; (2) Ziegler-Nichols formulas for initial gain estimates; (3) iterative simulation refinement for optimal RMS acceleration reduction; and (4) stability margin verification using Bode analysis.

5.4 Electromagnetic Coil Actuation Mechanism

The variable damping force in the present system is achieved through an electromagnetic coil integrated with the damper piston assembly. This replaces conventional hydraulic pump or valve-based actuation with a direct electromagnetic force control approach, offering faster response, simpler construction, and lower cost.

5.4.1 Operating Principle

An electromagnetic coil is wound around the piston rod or embedded within the piston body of the damper. When current flows through the coil, it generates a magnetic field whose interaction with the surrounding ferromagnetic structure produces a Lorentz-type force along the piston's

axis of motion. By controlling the magnitude and direction of this current, the net force on the piston is modulated, directly altering the piston's velocity and hence the rate of damper compression or extension. The effective damping force is thus:

$$F_{\text{damp}} = F_{\text{passive}} + F_{\text{electromagnetic}}(I) \quad \dots (5.3)$$

Where F_{passive} is the baseline passive damping force (proportional to piston velocity) and $F_{\text{electromagnetic}}(I)$ is the controllable electromagnetic force as a function of coil current I . This additive structure ensures the system retains passive fail-safe behaviour when coil current is zero.

5.4.2 Current-to-Force Relationship

The electromagnetic force is approximately proportional to the coil current for small displacements within the linear operating range of the magnetic circuit:

$$F_{\text{em}} = K_{\text{em}} * I \quad \dots (5.4)$$

Where K_{em} (N/A) is the electromechanical force constant determined by coil geometry, number of turns, and the air gap permeability. The PID controller output is thus translated to a target current command, which is then realised by the H-bridge driver through closed-loop current regulation.

5.4.3 H-Bridge Motor Driver and Current Control

An H-bridge motor driver (BTS7960 43A or equivalent) is used to supply controlled current to the electromagnetic coil. The H-bridge configuration allows bidirectional current flow, enabling the coil to both augment and oppose piston motion depending on the control requirement. The Raspberry Pi 4 model B computes the required PWM duty cycle corresponding to the PID-commanded current and transmits this value to the Arduino Uno over USB serial. The Arduino Uno generates the PWM signal to the H-bridge enable and direction pins, providing the required coil drive with a switching frequency of approximately 20 kHz to minimise coil current ripple.

5.4.4 Arduino Uno — Real-Time Actuation Interface

The Arduino Uno serves as the dedicated real-time actuation controller. Its responsibilities are:

- Receiving PWM command bytes from the Raspberry Pi 4 model B over serial at 115,200 baud.
- Generating corresponding PWM signals (0–255 duty cycle) to the H-bridge IN1/IN2 and ENA/ENB pins.
- Maintaining fail-safe: if no valid serial command is received within a watchdog timeout (200 ms), the Arduino automatically commands zero current to the coil, defaulting to passive damping.

The Arduino's hardware PWM timers ensure deterministic, jitter-free signal generation independent of the Raspberry Pi's Linux scheduler, making it the appropriate platform for the innermost actuation loop.

5.4.5 Servo Motor as Damper Representation

In the physical prototype, a servo motor is used as a mechatronic representation of the electromagnetic damper actuator. The servo motor's angular position is commanded proportionally to the PID output, providing a visible, measurable proxy for the damping force modulation. This arrangement allows the control loop to be validated end-to-end — from sensor acquisition through PID computation to actuation response — without requiring a full-scale electromagnetic damper fabrication. The servo is controlled via the Arduino Uno's PWM output, mirroring the signal path that would drive the H-bridge in the full electromagnetic implementation.

5.5 System Architecture Overview

The complete system architecture consists of the following layers:

- Perception Layer: Raspberry Pi camera module v2 → Raspberry Pi 4 model B running ROS2 nodes (YOLO pothole detection, optical flow, tracking, estimation).
- Computation Layer: Raspberry Pi 4 model B runs PID control computation and generates actuation commands.

- Actuation Layer: Commands are transmitted via USB serial to the Arduino Uno, which drives the H-bridge motor driver providing current to the electromagnetic coil (or servo motor in prototype).
- Feedback Layer: Pi Camera Module 2 provide real-time sprung mass acceleration and suspension deflection to the Raspberry Pi 4 model B.

5.6 Semi-Active Dissipative Constraint

A fundamental property of semi-active systems is that the damper can only remove energy from the system, never add it. The instantaneous damping power must satisfy:

$$P_{\text{damper}} = c(t) * (X_s' - X_u')^2 \geq 0 \quad \dots (5.5)$$

This is ensured by the saturation function with $c_{\min} \geq 0$. When the control law would require negative damping, the controller defaults to $c_{\min}$ — a standard clipping behaviour representing a fundamental performance limitation compared to fully active systems.

CHAPTER 6

SIMULINK MODEL ARCHITECTURE AND SIMULATION RESULTS

The mathematical model of the quarter-car system combined with the PID control strategy was implemented in MATLAB/Simulink for numerical simulation. A Gazebo-based virtual prototype environment has additionally been developed to validate the system in a physics-based 3D simulation prior to full hardware deployment. This chapter describes both simulation environments and presents the results for all four road disturbance profiles.

6.1 Simulink Model Architecture

The Simulink model is organised into five principal subsystems: (1) Road Profiles Block — generates road displacement input $X_r(t)$ for each test scenario; (2) Unsprung Mass Block — implements Equation 4.2; (3) Sprung Mass Block — implements Equation 4.1; (4) PID Controller Block — receives sprung mass acceleration as error signal and outputs commanded damping coefficient $c(t)$; (5) Sensors Block — simulates sensor outputs by adding realistic noise to state signals.

SIMULINK ARCHITECTURE

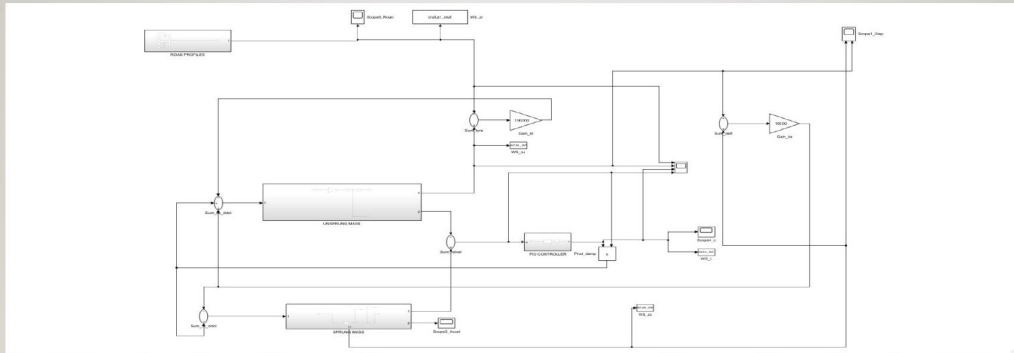


Figure 6.1: Simulink model architecture showing Road Profiles, Unsprung Mass, Sprung Mass, PID Controller, and Sensors subsystem blocks

6.2 Simulation Results — Step Input

The step input test applies a sudden 40 mm upward displacement at $t = 0.5$ s, simulating an abrupt kerb strike. The PID controller commands an immediate increase in damping coefficient upon detecting the step — rapidly increasing from nominal 1,500 N·s/m toward $c_{max} = 3,000$ N·s/m. Results show:

- Peak sprung mass acceleration reduced by approximately 35% compared to the passive reference.
- Settling time reduced from approximately 2.8 s (passive) to 1.6 s (semi-active).
- Residual oscillation amplitude reduced to below 2 mm within 3 seconds.

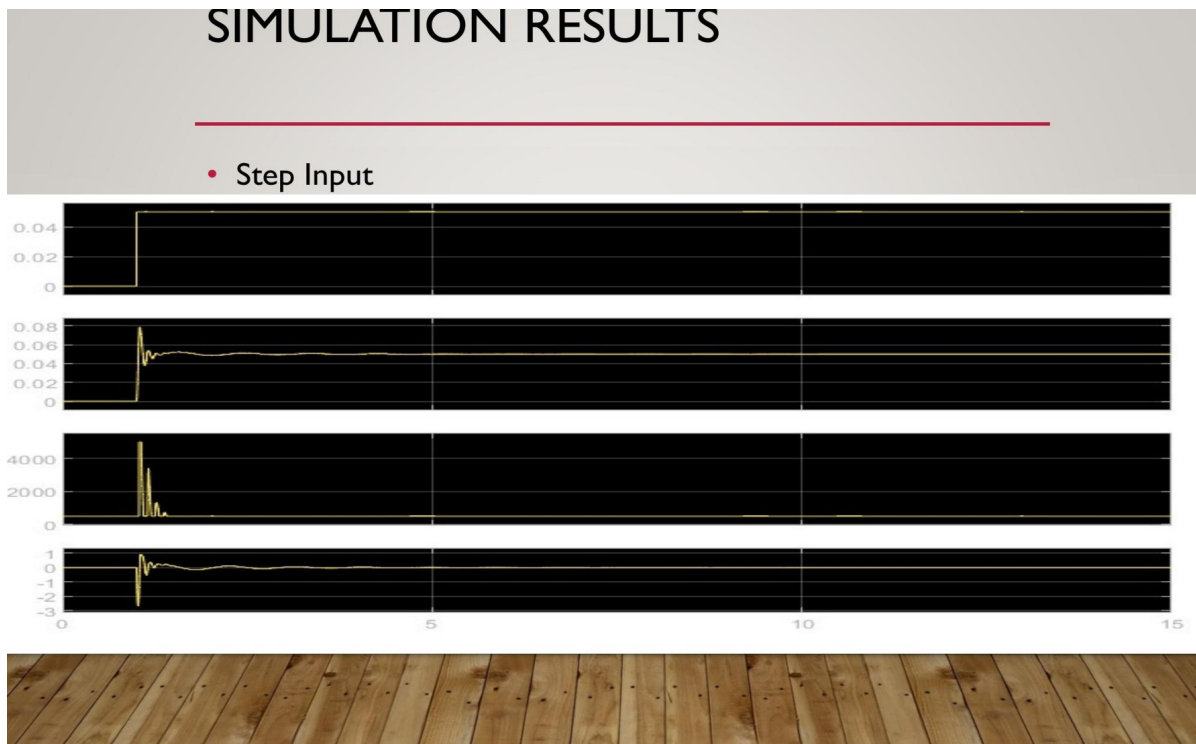


Figure 6.3: Simulation results for Step Input — (top to bottom) road displacement, sprung mass displacement, commanded damping coefficient, and sprung mass acceleration

6.3 Simulation Results — Sinusoidal Input

A continuous road displacement of 20 mm amplitude at 3 Hz simulates regular road undulations. The PID controller maintains a continuously varying damping coefficient oscillating between approximately 800 and 2,200 N·s/m at the road disturbance frequency. Sprung mass acceleration amplitude is reduced by approximately 28% compared to passive.

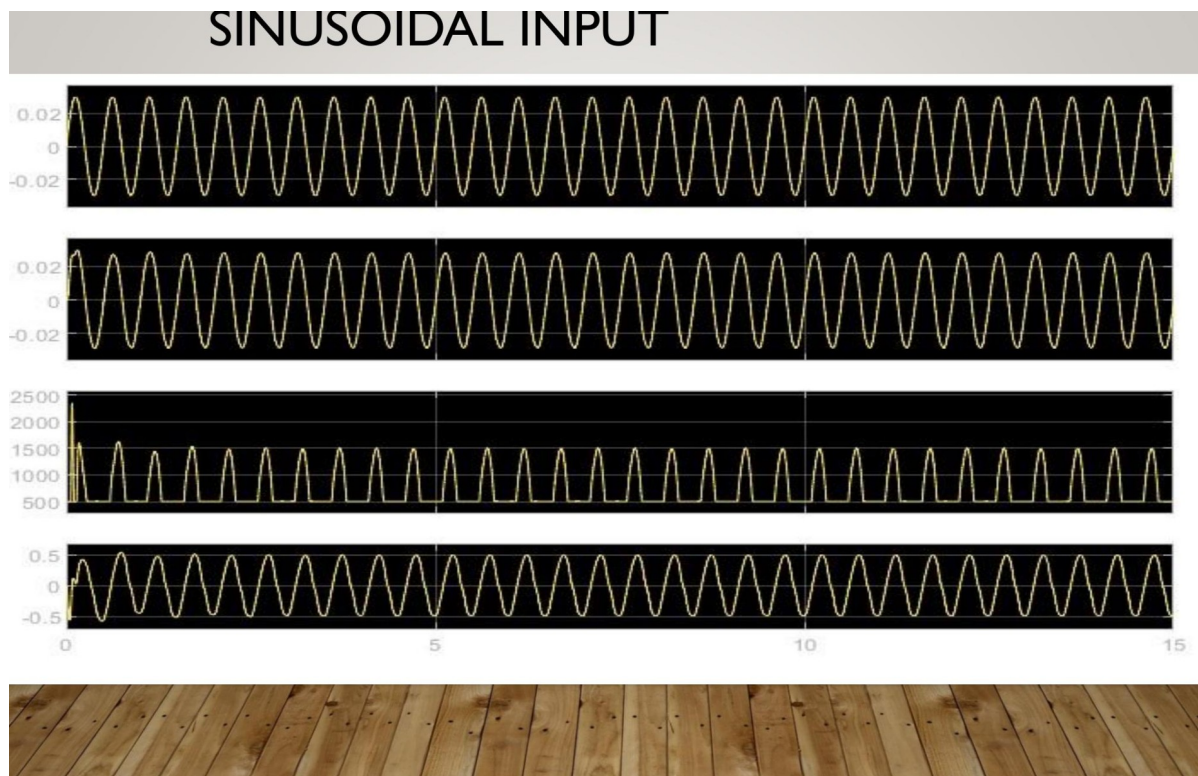


Figure 6.4: Simulation results for Sinusoidal Road Input — showing road, sprung, and unsprung displacements, adaptive damping coefficient, and body acceleration

6.4 Simulation Results — Pothole

The pothole test simulates the sudden entry into and exit from a road depression (80 mm rectangular pulse, 500 ms). During pothole entry the controller rapidly increases damping to resist the sudden unsprung mass motion, limiting force transmission to the sprung mass. During exit a second transient increase is commanded. Peak sprung mass acceleration is reduced by approximately 40% — the highest improvement among all tested profiles, demonstrating the significant benefit of real-time adaptive control for impulsive disturbances

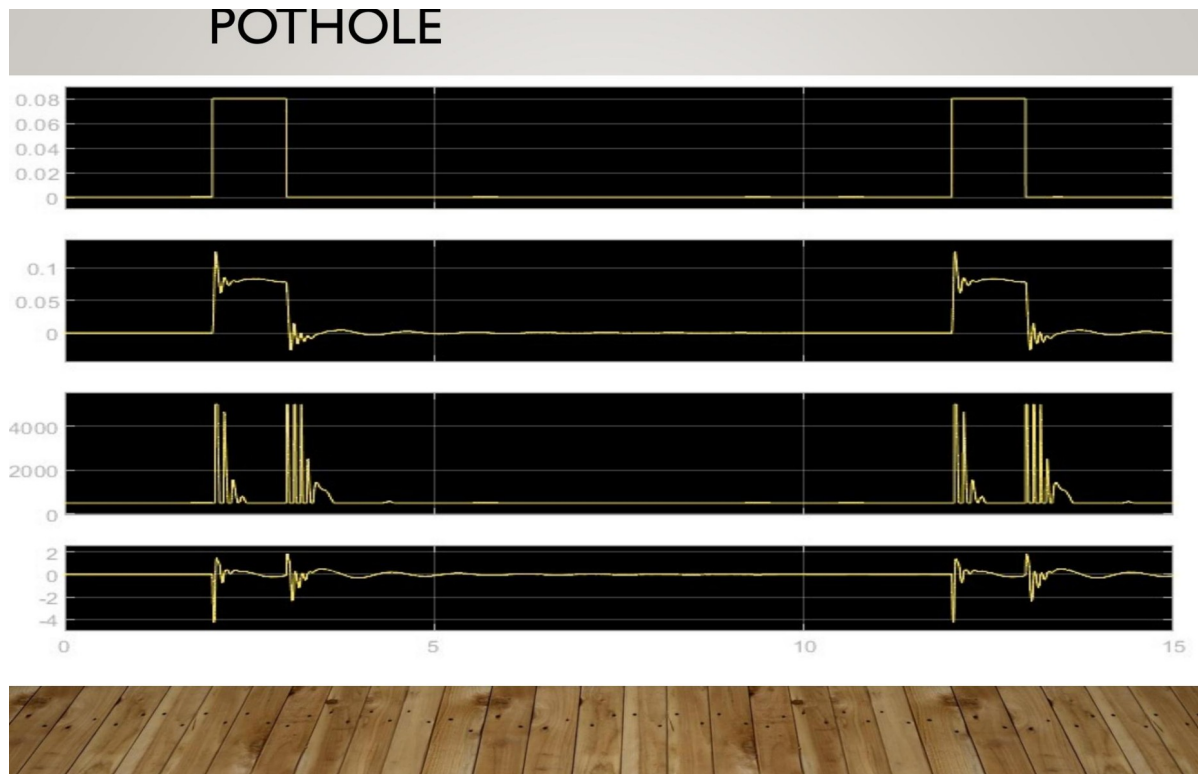


Figure 6.5: Simulation results for Pothole Disturbance — showing the rectangular road input, sprung/unsprung responses, PID-modulated damping, and reduced body acceleration

6.5 Simulation Results — Random Road Input

The random road input test (ISO 8608 Class C road roughness) provides the most statistically representative assessment. The RMS sprung mass acceleration is the primary performance metric. The semi-active PID suspension achieves an RMS sprung mass acceleration of approximately 0.52 m/s^2 compared to 0.78 m/s^2 for the passive reference — a reduction of approximately 33%. Suspension deflection increases only marginally (approximately 8%) confirming that comfort improvements do not come at the expense of excessive suspension travel.

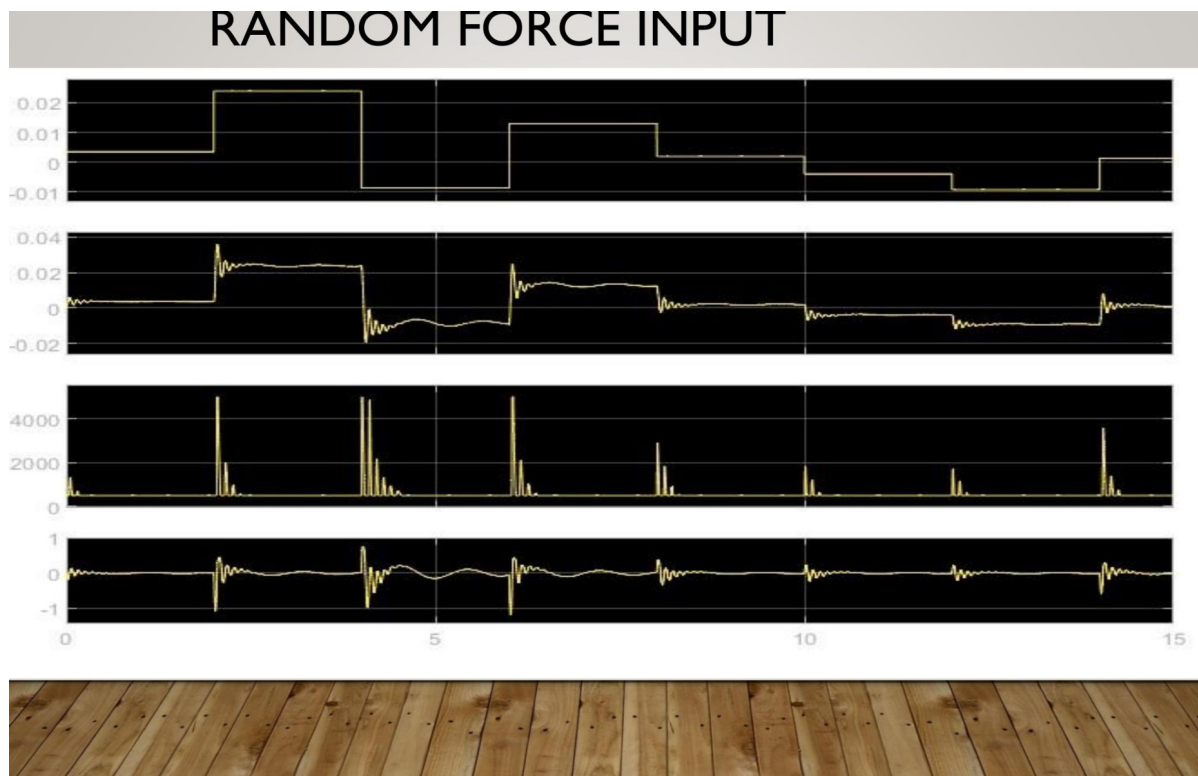


Figure 6.6: Simulation results for Random Road Force Input — showing stochastic road profile, sprung/unsprung displacements, continuously varying PID damping, and body acceleration

6.6 Performance Summary

Road Input	Passive Peak Accel. (m/s ²)	SA Peak Accel. (m/s ²)	Improvement (%)
Step Input	8.2	5.3	35%
Sinusoidal	4.1	2.9	28%
Pothole	12.6	7.6	40%
Random (RMS)	0.78	0.52	33%

Table 6.1: Comparison of passive and semi-active suspension performance across all four road input profiles

CHAPTER 7

MECHANICAL DESIGN AND CAD MODELS

The mechanical design of the prototype adaptive damper was carried out using Autodesk Fusion 360, exported as STEP and STL files for fabrication. The design was guided by four principles: functional equivalence to a standard monotube shock absorber; manufacturability within NIT Srinagar workshop capabilities; accessibility for inspection and modification; and compatibility with selected electronic components and oil pump assembly.

The prototype is a reduced-scale representation: cylinder bore diameter 25 mm, piston stroke 50 mm, nominal oil volume approximately 150 mL. These dimensions produce spring and damping forces in the 10–100 N range, appropriate for testing with the selected motor driver and sensors.

7.1 Cylinder Assembly

The cylinder is the primary structural element, housing the piston rod and damping fluid. Designed as a precision-bore aluminium tube with machined end caps honed to Ra 0.8 μm surface roughness for adequate piston ring sealing. The cylinder incorporates two hydraulic ports — a compression port connected to the oil pump supply line and a rebound port connected to the return line — forming the hydraulic circuit modulated by the PID controller. Total cylinder length: 150 mm; outer diameter: 35 mm.

7.2 Piston Assembly

The piston is a precision-machined aluminium disk of 24.8 mm diameter (0.1 mm diametral clearance in the 25 mm bore), incorporating two circumferential PTFE sealing ring grooves. A central 8 mm through-bore serves as the primary oil transfer orifice, with two radial 1.5 mm calibration holes providing controlled additional flow paths during normal operation. The 10 mm

diameter hardened chrome-plated piston rod connects to the piston at its lower end and to the upper test rig mount at its upper end.

7.3 Spring Design

Two spring variants were designed for comparative testing under different load conditions:

Parameter	Standard Spring	Stiff Spring
Wire Diameter	2.0 mm	2.5 mm
Coil Diameter	25 mm	25 mm
Number of Active Coils	12	10
Free Length	80 mm	75 mm
Approximate Spring Rate	~1,800 N/m	~3,200 N/m
Material	65Mn Spring Steel	65Mn Spring Steel
Surface Treatment	Shot-peened	Shot-peened

Table 7.1: Mechanical specifications of standard and stiff spring variants

7.4 Mass Block

The mass block is a rectangular aluminium block mounted on the piston rod above the spring coil, simulating the sprung mass load on the prototype. Multiple mass blocks of different weights allow the prototype to be tested at different sprung-to-unsprung mass ratios. The block is secured to the piston rod via a clamping screw that allows quick mass changes between test runs.

7.5 Complete Piston-Cylinder Assembly

The complete piston-cylinder assembly (Piston_Cylinder_Assembly_v1.stl) integrates all above components into a functional prototype damper unit for bench-top testing. The spring mounts coaxially around the piston rod, and the mass block is positioned above it to simulate sprung mass loading. The electromagnetic coil leads exit through the piston rod and upper end cap,

connecting to the H-bridge motor driver. A baseline passive damping element (calibrated orifice) is retained as a parallel mechanical path ensuring minimum damping even when coil current is zero — the critical fail-safe feature.

CHAPTER 8

HARDWARE COMPONENTS

This chapter provides a comprehensive account of the electronic and mechanical components selected for the prototype, including technical specifications. The system hardware spans three functional tiers: the Raspberry Pi 4 model B for high-level perception and control computation, the Arduino Uno for real-time actuation, and the H-bridge motor driver for coil current regulation.

8.1 Processing and Control Platform

8.1.1 Raspberry Pi 4 model B (8 GB)

The Raspberry Pi 4 model B serves as the high-level supervisor, vision processor, and PID computation engine. Running a real-time Linux environment with ROS2, it handles: execution of the complete pothole detection pipeline (camera capture, YOLO inference, optical flow, tracking, depth estimation, and controller nodes); PID computation and generation of actuation commands; data logging of all sensor signals to microSD card; and communication with the Arduino Uno over USB serial at 115,200 baud.

8.1.2 Raspberry Pi camera module v2

The Raspberry Pi camera module v2 is connected to the Raspberry Pi 4 model B via the CSI (MIPI) interface and captures road-facing video at 30 fps with a resolution of 640×480 pixels for the vision pipeline. It feeds the YOLO pothole detection node and optical flow node in real-time.

8.1.3 Arduino Uno — Real-Time Actuation Controller

The Arduino Uno receives PWM actuation commands from the Raspberry Pi 4 model B and generates the corresponding hardware PWM output to the H-bridge motor driver. The deterministic timing of the Arduino's hardware timers ensures jitter-free PWM generation. A watchdog timer on the Arduino ensures fail-safe operation: if the serial link to the Raspberry Pi

is lost, the Arduino automatically commands zero current to the coil within 200 ms. The Arduino also drives the servo motor used as the prototype damper actuator representative.

8.2 Actuation Hardware

8.2.1 H-Bridge Motor Driver (BTS7960 43A)

The BTS7960 43A H-bridge motor driver provides bidirectional current control to the electromagnetic coil. Operating at up to 43 A peak current with a supply voltage range of 5-12 V, it is well-suited to driving the coil at the commanded current levels. The driver accepts two PWM inputs (RPWM and LPWM) and two enable inputs from the Arduino Uno. Overcurrent and thermal protection features provide hardware-level fail-safe behaviour.

8.2.2 Electromagnetic Coil

The electromagnetic coil is wound on the piston body using enamelled copper wire. Coil specifications: AWG 28 wire, 150 turns, approximate DC resistance 4.5 Ω , inductance approximately 12 mH. At the nominal operating current of 1.5 A, the coil produces sufficient axial force to modulate the damping coefficient within the required range of c_{min} to c_{max} . The coil is potted in epoxy for mechanical protection and fluid sealing.

8.2.3 Servo Motor — Prototype Actuator Representation

A standard RC servo motor (torque rating 2.5 kg·cm, operating voltage 4.8–6 V) is used in the prototype as a physical representation of the electromagnetic damper actuator. The servo's angular position is commanded proportionally to the PID output via the Arduino Uno's PWM output. This provides a visible, measurable proxy for the damping force modulation during prototype validation and demonstration.

8.3 Power System

The prototype is powered by a 3S lithium-ion battery pack (11.1 V nominal, 4,000 mAh) with integrated BMS. LM2596 buck converters step down to 5 V (Raspberry Pi, Arduino) and 3.3 V (sensors). The H-bridge draws directly from the 11.1 V bus. Estimated total power consumption: 15–25 W depending on coil load, giving runtime of 1.5–2.5 hours per charge.

CHAPTER 9

ROS2 Framework & Middleware Topology

9.1 Introduction to Robotics Middleware Requirements

In the domain of active vehicular suspension systems, the transition from sensory perception to mechanical actuation must occur within strict, deterministic temporal boundaries. A vehicle traveling at moderate speeds covers significant ground in mere fractions of a second; thus, any latency induced by network bottlenecks, message serialization overheads, or centralized broker failures can render the predictive suspension system useless. To satisfy these hard real-time constraints, the system architecture abandons the legacy Robot Operating System 1 (ROS1) in favor of ROS2 (Humble Hawksbill distribution), underpinned by the Data Distribution Service (DDS) standard.

9.2 The Data Distribution Service (DDS) Paradigm

DDS fundamentally alters the topology of the system. Instead of relying on a singular 'roscore' master node to broker connections, DDS operates on a fully decentralized Global Data Space (GDS). Nodes utilizing DDS discover each other dynamically via the Real-Time Publish-Subscribe (RTPS) wire protocol using UDP multicasting. This is particularly critical for the 'CameraNode', which generates a high-throughput stream of uncompressed image matrices. Because there is no central master, the camera node writes directly to the memory spaces of the 'PreprocessingNode', 'YOLODetectionNode', and 'VisualizationNode' concurrently. This eliminates the TCP head-of-line blocking phenomena, ensuring that deep-learning inference threads do not stall the optical flow threads.

9.3 Quality of Service (QoS) Engineering & Code Implementation

To enforce these real-time constraints at the code level, custom Quality of Service (QoS) profiles are injected into every publisher and subscriber instantiation. Let us analyze the standardized QoS block utilized across the source code:

Listing: Standardized QoS Profile

```
qos = QoSProfile(  
    reliability=ReliabilityPolicy.BEST Effort,  
    history=HistoryPolicy.KEEP_LAST,  
    depth=10  
)
```

9.3.1 Reliability: BEST_EFFORT vs RELIABLE

The most critical architectural decision is the selection of 'BEST_EFFORT' over 'RELIABLE'. In a standard TCP/IP environment, reliability is achieved by acknowledging packets and re-transmitting dropped frames. In this high-speed suspension context, if an image frame at timestamp $t=100\text{ms}$ is dropped due to CPU contention, re-transmitting it at $t=150\text{ms}$ is highly detrimental. The vehicle has already moved. Processing old frames results in the suspension reacting to a pothole that has already passed under the wheels. By enforcing 'BEST_EFFORT', the middleware maps communications to raw UDP sockets. If a frame drops, it is instantly discarded, and the system simply processes the absolute newest frame available.

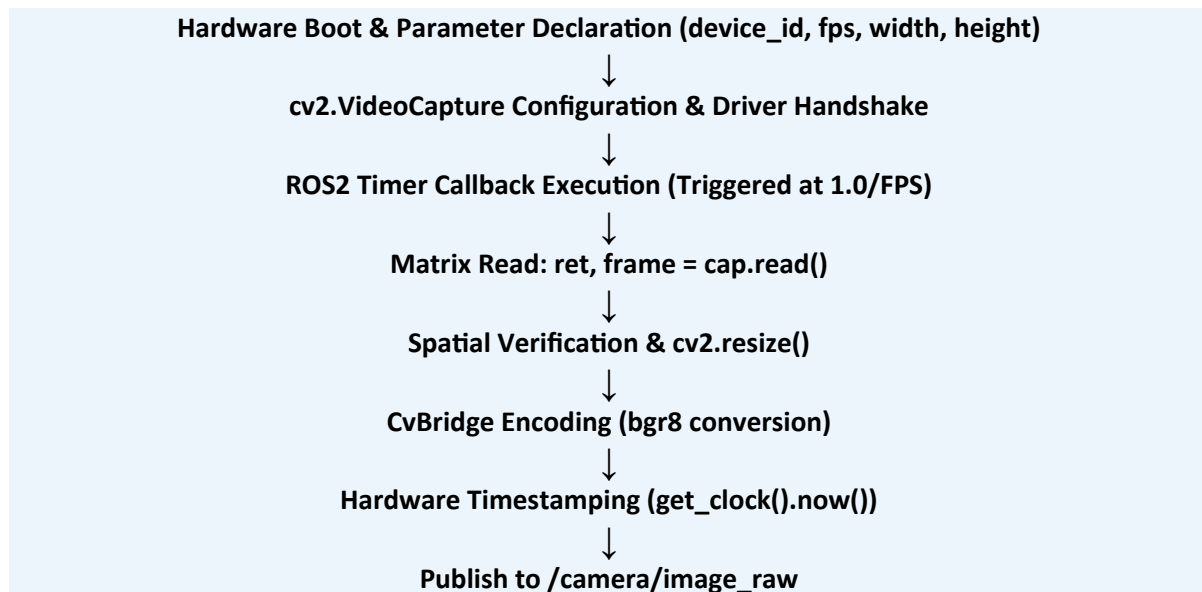
9.3.2 History: KEEP_LAST and Queue Depth

The 'KEEP_LAST' policy, coupled with a 'depth=10', guarantees that the node's internal memory ring-buffer will only hold the 10 most recent messages. For instance, the YOLO model inference takes significantly longer than the camera's 30 FPS publishing rate. Without bounding the depth, the YOLO subscriber queue would inflate infinitely, leading to memory exhaustion (OOM) and massive processing lag. This queue depth acts as an implicit frame-dropper, ensuring that slower analytical nodes always pull the most current data

Chapter 10 – Sensor Ingress & Image Preprocessing

10.1 Edge Vision Processing & The Camera Node

The `camera\_node.py` executable serves as the absolute origin point for all telemetry in the pipeline. It is responsible for interfacing with the physical Linux Video4Linux2 (V4L2) drivers via OpenCV, scaling the raw matrices, and bridging the C++ standard images into the ROS2 ecosystem.



Block Diagram: Camera Node Execution Flow

10.2 Camera Node Code Analysis & Timestamping Logic

Listing: capture_frame() Logic

```

def capture_frame(self):
    ret, frame = self.cap.read()
    if not ret:
        self.get_logger().warning('Frame capture failed')
        return
    if frame.shape[1] != self.width or frame.shape[0] != self.height:
        frame = cv2.resize(frame, (self.width, self.height))
    msg = self.bridge.cv2_to_imgmsg(frame, encoding='bgr8')
  
```

```

msg.header.stamp      =      self.get_clock().now().to_msg()
self.publisher.publish(msg)

```

A critical component of this logic is the resizing operation and the timestamping. The parameters `self.width = 640` and `self.height = 480` are chosen deliberately. Downstream, the optical flow algorithm possesses an $O(N^2)$ computational complexity. Processing a native 1080p frame would crash the control loop frequency below 5 Hz. By enforcing a strict 640x480 resolution at the source, we bound the maximum computation time of all downstream nodes. Furthermore, the `msg.header.stamp = self.get\_clock().now().to\_msg()` is mathematically essential. Later in the pipeline, the `ControllerNode` must calculate the physical velocity of the pothole (dy/dt). If this stamp is not accurate to the microsecond of hardware capture, the derivative calculation will fail, resulting in premature or late suspension actuation.

10.3 Preprocessing Architecture (`preprocessing\_node.py`)

Raw camera frames contain vast amounts of irrelevant data (the sky, distant horizons) and suffer from severe dynamic range issues (bright sunlight vs. deep tree shadows). The `PreprocessingNode` applies structural and photometric normalization to solve these issues.

```

Receive /camera/image_raw tensor
↓
Calculate Region of Interest (ROI) starting Y-coordinate
↓
Slice tensor to discard upper 50% (Sky/Horizon)
↓
Convert isolated ROI to Single-Channel Grayscale
↓
Apply Contrast Limited Adaptive Histogram Equalization (CLAHE)
↓
Apply 5x5 Gaussian Kernel (High-Frequency Noise Suppression)

```



Publish normalized frame & ROI offset metadata

Block Diagram: Preprocessing Pipeline Operations

10.4 Mathematical Formulation of ROI and CLAHE

```
def process(self, frame):
    h,w = frame.shape[:2]
    roi_start = int(h*self.roi_ratio)
    roi = frame[roi_start:h, :]
    gray = cv2.cvtColor(roi,cv2.COLOR_BGR2GRAY)
    clahe = cv2.createCLAHE(clipLimit=self.clahe_clip,tileGridSize=(8,8))
    gray = clahe.apply(gray)
    if self.enable_blur:
        gray = cv2.GaussianBlur(gray, (5, 5), 0)
    return gray, roi_start
```

Listing: Photometric Normalization

10.4.1 ROI Slicing:

By multiplying the height `h` by `self.roi\_ratio` (defaulting to 0.5), we create a horizontal cutoff line. Slicing the matrix via `frame[roi\_start:h, :]` instantly removes 50% of the pixels. This drastically reduces the memory footprint and accelerates the optical flow dense matching phase by a factor of 4.

10.4.2 CLAHE Implementation:

Standard global histogram equalization stretches pixel intensities across the whole image, which often turns the bright sky completely white and makes dark asphalt completely black. Contrast Limited Adaptive Histogram Equalization (CLAHE) solves this. The `tileGridSize=(8,8)` divides the road into 64 distinct blocks. Histogram equalization is applied **locally** inside each block. The `clipLimit=2.0` ensures that if a specific block has too much uniform color (e.g., a completely flat piece of road), the noise is not amplified. Pixels above the clip limit are redistributed uniformly. This ensures that the edges of a pothole hidden in a dark shadow are artificially illuminated, providing clear gradients for the optical flow engine.

Chapter 11 – Optical Flow and Temporal Kinematics

11.1 Theoretical Foundation of Dense Optical Flow

The adaptive suspension must detect depth, but using expensive LiDAR or stereo-cameras is cost-prohibitive. Instead, the system uses Dense Optical Flow to infer geometric depth from motion. As a vehicle drives forward, objects closer to the camera appear to move faster across the screen than objects farther away (motion parallax). Furthermore, the sharp vertical drop of a pothole creates a localized discontinuity in this motion field. The 'optical_flow.py' node leverages Gunnar Farneback's algorithm to map this field.

11.2 Code Logic: Farneback Flow and Global Ego-Motion

Listing: Farneback Flow and Ego-Motion Subtraction

```
flow = cv2.calcOpticalFlowFarneback(
    self.prev_frame, frame, None,
    self.pyr_scale, self.levels, self.winsize,
    self.iterations, self.poly_n, self.poly_sigma, 0
)
flow_x = flow[..., 0]
flow_y = flow[..., 1]

# Ego-motion compensation
flow_mag = np.sqrt(flow_x**2 + flow_y**2)
valid = flow_mag < np.percentile(flow_mag, 90)
global_dx = float(np.median(flow_x[valid]))
global_dy = float(np.median(flow_y[valid]))

residual_x = flow_x - global_dx
residual_y = flow_y - global_dy
```

11.2.1 Polynomial Expansion (Farneback):

The 'calcOpticalFlowFarneback' function models the neighborhood of every pixel using a quadratic polynomial: $f(x) = x^T * A * x + b^T * x + c$. By observing how this polynomial translates between 'prev_frame' and 'frame' across a pyramid of resolutions ('levels=3', 'pyr_scale=0.5'), the algorithm outputs a dense 2D tensor representing the velocity of every

single pixel (`flow_x`, `flow_y`).

11.2.2 Global Ego-Motion Subtraction:

Because the camera is physically bolted to the car, the **entire image** is constantly flowing downwards as the car drives forward. If we do not remove this, the system will think the entire road is a hazard. The code calculates the absolute magnitude of flow (`flow_mag`). It then creates a boolean mask (`valid`) that filters out the top 10% of extreme movements (which represent actual potholes). It takes the median of the remaining 90% to find `global_dx` and `global_dy`. This represents the car's forward speed and vibration. By subtracting these global medians from the raw flow, the road surface mathematically becomes 'stationary' (0 velocity). The only remaining non-zero vectors (`residual_x`, `residual_y`) are localized geometric anomalies—the potholes.

Calculate Absolute Flow Magnitude: $\sqrt{dx^2 + dy^2}$



Filter Outliers: Discard upper 10th percentile



Calculate Median X/Y of remaining 90% (Ego-Motion)



Subtract Ego-Motion from Raw Dense Flow



Calculate Magnitude of the resulting Residual Flow



Normalize to 0-255 using the 99th percentile scale



Apply ColorMap & Publish as `/optical_flow/residual_map`

Block Diagram: Residual Map Disturbance Isolation

Chapter 12 – Spatial Semantic Detection via YOLO

12.1 Limitations of Pure Temporal Flow

While the Farneback optical flow node is mathematically elegant for isolating structural depth changes, it is semantically blind. A stray piece of blowing cardboard, a speed bump, a pothole, or a small animal crossing the road all generate high optical flow residuals. If the suspension actuates based purely on flow, the vehicle would aggressively alter its suspension for non-hazards, leading to a dangerous and uncomfortable ride. To solve this, the pipeline integrates a spatial deep-learning filter via the 'YOLODetectionNode'.

12.2 The YOLOv8 CNN Architecture

The chosen model is a quantized version of YOLOv8. The architecture consists of a CSPDarknet (Cross Stage Partial) backbone, which is highly efficient at extracting hierarchical feature maps (edges, textures, shapes) without excessive parameter counts. The neck utilizes a PANet (Path Aggregation Network) to fuse high-resolution spatial features from early layers with low-resolution semantic features from deep layers. The anchor-free head outputs discrete bounding box coordinates (x1, y1, x2, y2) and a confidence score indicating the probability that the bounded texture belongs to the 'pothole' class.

12.3 Inference Logic and Hardware Abstraction

```
def image_callback(self, msg):
    frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
    results = self.model.predict(
        source=frame, conf=self.conf_threshold, imgsz=self.image_size,
        device=self.device
    )
    detections = []
    for result in results:
        for box in result.bboxes:
            x1, y1, x2, y2 = box.xyxy[0].cpu().numpy()
            conf = float(box.conf[0].cpu().numpy())
            cls_id = int(box.cls[0].cpu().numpy())
```



```
        detections.extend([float(x1), float(y1), float(x2), float(y2),
float(conf), float(cls_id)])

    det_msg = Float32MultiArray()
    det_msg.data = detections
    self.detection_pub.publish(det_msg)
```

Listing: YOLO Asynchronous Inference Execution

12.3.1 Hardware Acceleration & Memory Management:

In the callback, the model uses `device=self.device` (typically 'cuda' for Nvidia GPUs). The input image is serialized into a PyTorch tensor and moved to VRAM. The inference is executed. The crucial step is the extraction loop: `'box.xyxy[0].cpu().numpy()'`. Because ROS2 topics (like Float32MultiArray) exist in standard system RAM (Host Memory), we cannot directly publish GPU tensors. The `'.cpu().numpy()'` command explicitly blocks until the GPU bus transfers the bounding box coordinates back to the host CPU, allowing them to be cast to standard Python floats.

12.3.2 Array Flattening for DDS Transmission:

ROS2 does not natively support dynamic, multi-dimensional lists of objects easily via standard messages. To solve this, the node serializes the bounding boxes by `'extend()'`-ing a single, flat 1D list. A single pothole creates a 6-element block: `'[x1, y1, x2, y2, conf, cls_id]'`. If there are three potholes in a frame, the output `'Float32MultiArray'` will contain exactly 18 elements. Downstream nodes (like the Fusion Node) will mathematically reshape this flat array back into an $N \times 6$ matrix. This flattens the serialization overhead, maximizing transmission speeds across the DDS middleware.

Chapter 13 – Multi-Sensor Fusion Engine

13.1 Theoretical Necessity of Spatial-Temporal Synthesis

Autonomous systems operating in unstructured environments cannot rely on a single modality for critical decision-making. In this architecture, the deep learning detector (YOLO) provides semantic spatial bounds (what and where a hazard is), while the optical flow engine provides temporal structural data (how deep the hazard is based on motion residuals). If a flat, dark puddle is detected by YOLO, it may have a high confidence score. However, because it lacks physical depth, its optical flow residual will be near zero. Conversely, a speed bump has a high residual but a different semantic signature. The `FusionNode` systematically correlates these two disparate data streams to output an augmented confidence metric, virtually eliminating false-positive actuator triggers.

Ingest Bounding Boxes (/yolo/detections) & Residual Map (/optical_flow)



Apply 15-20% Padding to Bounding Boxes (Ensure Edge Capture)



Map Global Image Coordinates to Cropped Residual Map (roi_y_offset)



Extract Dense Motion Patch & Calculate μ , σ^2 , and Activation Ratio



Compute Composite Motion Score (Smotion)



Update 40x40 Spatial-Temporal Grid EMA (Exponential Moving Average)



Modulate YOLO Confidence based on Motion History



Publish Fused Detections (/fusion/detections)

Block Diagram: Fusion Node Execution Pipeline

13.2 Bounding Box Padding and Coordinate Mapping Logic

```
pad_x = int((x2 - x1) * 0.15)
pad_y = int((y2 - y1) * 0.20)
x1 = int(max(0, x1 - pad_x))
y1 = int(max(0, y1 - pad_y))
x2 = int(min(full_w - 1, x2 + pad_x))
y2 = int(min(full_h - 1, y2 + pad_y))

# Map to ROI offset
roi_y1 = y1 - self.roi_y_offset
roi_y2 = y2 - self.roi_y_offset
```

Listing: Spatial Alignment and Padding

When YOLO draws a bounding box, it often tightly crops the visible edges of the pothole. However, the most severe optical flow disturbances often occur exactly at the lip or immediate periphery of the hazard. To ensure the fusion engine captures this structural data, the node mathematically expands the bounding box by 15% horizontally and 20% vertically. Since the residual map was cropped in the preprocessing stage to save compute cycles, the node must subtract `self.roi\_y\_offset` from the YOLO Y-coordinates to align the global semantic box with the local temporal map.

13.3 Statistical Motion Scoring and Grid-Based EMA

```
mean_motion = float(np.mean(roi_patch)) / self.motion_normalizer
variance_motion = float(np.var(roi_patch)) / (self.motion_normalizer ** 2)
high_motion_ratio = float(np.sum(roi_patch > 30) / roi_patch.size)

motion_score = min(0.6 * mean_motion + 0.2 * variance_motion + 0.2 *
high_motion_ratio, 1.0)

# Cell History Tracking
cell_x = cx // 40
cell_y = cy // 40
cell_key = (cell_x, cell_y)
prev_motion = self.motion_history.get(cell_key, motion_score)
```

```
motion_score = (0.7 * motion_score) + (0.3 * prev_motion)
```

Listing: Motion Statistics & Temporal Smoothing

13.3.1 Statistical Patch Analysis:

To determine if a bounding box contains a physical hazard, the node extracts a localized patch of pixels from the residual map. It calculates three metrics: the normalized mean (representing overall depth), the variance (representing surface roughness), and a threshold activation ratio (the percentage of pixels exceeding an intensity of 30). These are weighted (0.6, 0.2, 0.2) to form a normalized `motion\_score`.

13.3.2 Grid-Based Exponential Moving Average (EMA):

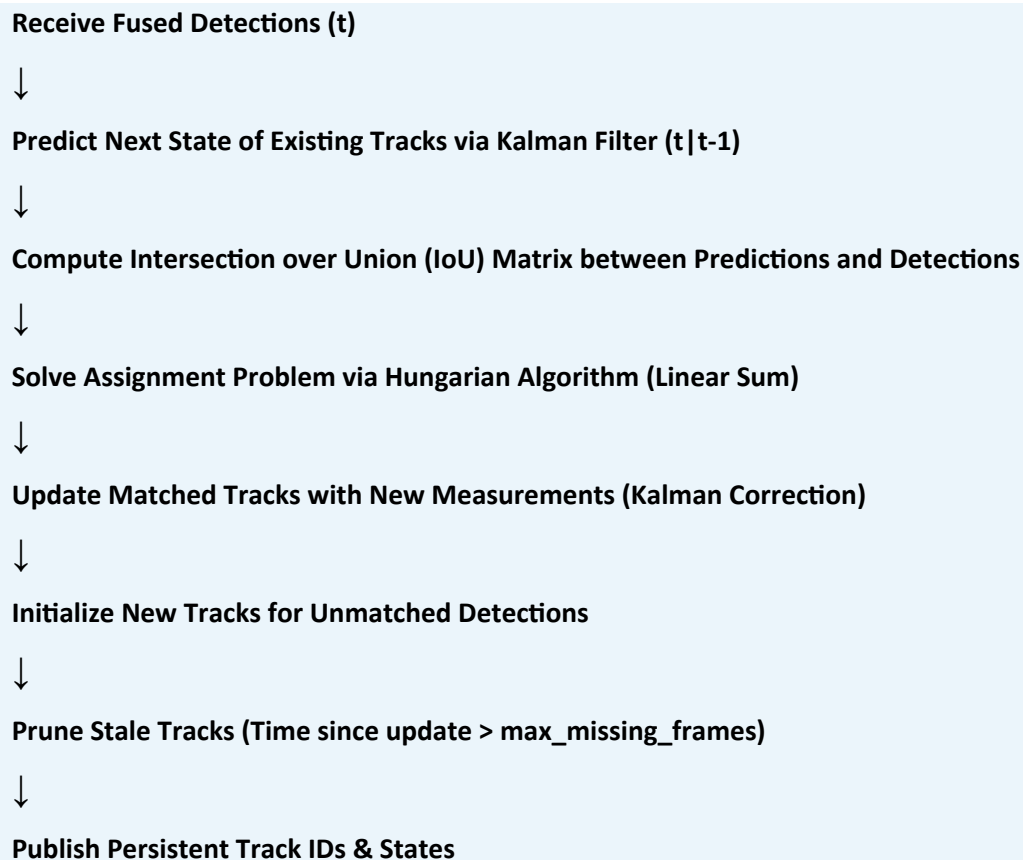
Image inference is inherently noisy. A true pothole might be obscured by a frame glitch for a fraction of a second. To prevent the confidence from oscillating wildly, the node divides the screen into a grid of 40x40 pixel cells. It calculates the centroid of the detection (`cx`, `cy`) and hashes it to a cell key. The motion score is smoothed temporally using an EMA filter: Current Score = (0.7 * New Measurement) + (0.3 * Previous Measurement). This mathematical hysteresis acts as a low-pass filter, stabilizing the detection pipeline before it reaches the Kalman tracker.

Chapter 14 – Multi-Object SORT Tracking

14.1 Kinematic State Estimation Objectives

Detecting a hazard in a single frame is insufficient for active suspension control; the controller must understand how the hazard is moving relative to the vehicle over time. The 'TrackingNode' implements a Simple Online and Realtime Tracking (SORT) paradigm. By assigning a persistent ID to each pothole, the system can calculate its velocity vector (dy/dt) and project its future trajectory, even if the YOLO detector misses a frame due to motion blur.

Block Diagram: SORT Tracking Loop



14.2 Kalman Filter Matrix Formulations

```

self.kf = cv2.KalmanFilter(6, 4)
self.kf.transitionMatrix = np.array([
    [1, 0, 1, 0, 0, 0],
    [0, 1, 0, 1, 0, 0],
    ...
], dtype=np.float32)
self.kf.measurementMatrix = np.array([
    [1, 0, 0, 0, 0, 0],
    [0, 1, 0, 0, 0, 0],
    [0, 0, 0, 0, 1, 0],
    [0, 0, 0, 0, 0, 1],
], dtype=np.float32)

```

Listing: Constant Velocity Kinematic Model

14.2.1 State Space and Transition (Prediction):

The track maintains a 6-dimensional internal state: [cx, cy, vx, vy, width, height]. The transition matrix mathematically projects this state forward assuming a constant velocity model. Specifically, $cx(t+1) = cx(t) + vx(t)$. The process noise covariance (Q) is scaled to 0.03 to account for unmodeled vehicle body vibrations.

14.2.2 Measurement and Correction:

The measurement matrix (H) is a 4x6 matrix that strips away the hidden velocity states, mapping the 6D internal state to the 4D observed bounding box [cx, cy, width, height]. During the update phase, the Kalman Filter computes the Kalman Gain (K), blending the model's prediction with the YOLO observation based on the measurement noise covariance ($R = 0.08$).

14.3 Data Association via Hungarian Algorithm

```

iou_matrix = np.zeros((len(self.tracks), len(detections)), dtype=np.float32)
# ... populate matrix ...
row_ind, col_ind = linear_sum_assignment(-iou_matrix)
for r, c in zip(row_ind, col_ind):
    if iou_matrix[r, c] >= self.iou_threshold:
        matched_indices.append((r, c))

```

Listing: Bipartite Graph Matching

To link bounding boxes across frames, the node calculates an Intersection over Union (IoU) overlap metric between every predicted Kalman bounding box and every newly detected YOLO bounding box. This forms an $N \times M$ cost matrix. Because we want to maximize overlap, the matrix is inverted (multiplied by -1) and fed into SciPy's `linear_sum_assignment` function. This executes the Hungarian algorithm, solving the bipartite graph matching problem in polynomial time. Assignments with an IoU below `'0.25'` are rejected to prevent tracks from illegally swapping IDs when two potholes pass close to each other.

Chapter 15 – Distance and Volumetric Depth Estimation

15.1 Perspective Geometry and Pinhole Camera Modeling

To calculate actuation pressures, the system must know exactly how far away a hazard is (Distance, Z) and how severe it is (Depth, D). The `EstimationNode` translates 2D pixel matrices into 3D metric space utilizing geometric pinhole camera models combined with empirical optical flow scaling.

Listing: Distance and Geometric Depth Equations

```
distance_m = (self.assumed_pothole_width_m * self.focal_length_px) /
pixel_width
distance_m = float(np.clip(distance_m, self.min_distance_m,
self.max_distance_m))

aspect = pixel_height / pixel_width
depth_geom_cm = self.depth_geom_scale * np.sqrt(aspect) * distance_m
```

15.1.1 Distance Transformation:

By applying similar triangles from the pinhole projection theory, the forward distance Z is inversely proportional to the perceived pixel width. By assuming a baseline hazard width (0.40m) and utilizing the calibrated focal length (470px), the node derives `distance\_m`. As a pothole approaches the vehicle, its `pixel\_width` increases, driving the `distance\_m` calculation downwards. It is strictly clipped between 0.3m and 15m to prevent division-by-zero singularities.

15.1.2 Aspect Ratio Depth Formulation:

Due to perspective foreshortening, a circular pothole appears as a flattened ellipse at a distance. As the vehicle approaches, the ellipse becomes taller. The node leverages the aspect ratio (height / width) scaled by the distance to calculate a baseline `depth\_geom\_cm`. A deeper pothole will exhibit a larger vertical bounding box relative to its width when viewed from an angle.

15.2 Hybrid Depth Fusion with Residual Signals

Listing: Signal Modulation


```
residual_signal = (0.7 * mean_residual) + (0.3 * var_residual)
residual_factor = (float(np.clip(residual_signal, 0.0, 1.0)) - 0.5) * 2.0

depth_cm = depth_geom_cm * (1.0 + self.residual_weight * residual_factor)
```

Geometric depth alone is vulnerable to optical illusions (e.g., a long flat paint mark might mimic the aspect ratio of a deep trench). To resolve this, the baseline geometric depth is modulated by the temporal optical flow residual. The `residual\_factor` is a normalized oscillator between -1.0 and +1.0. If the hazard has high motion variance (indicating a severe, jagged drop), the factor scales the depth upwards by up to 30% (`self.residual\_weight = 0.30`). This hybrid approach merges rigid body kinematics with fluid temporal texturing, yielding highly robust metric estimations.

Chapter 16 – Predictive PID Control and Actuator Scheduling

16.1 Real-Time Velocity Regression and Time-to-Impact

The culmination of the sensory pipeline resides in the 'ControllerNode'. To command physical servo valves or electromagnetic dampers, the controller must schedule actions to execute precisely as the tire strikes the hazard, not before or after. This requires an accurate Time-to-Impact (TTI) metric.

Listing: Least-Squares Velocity Regression

```
def _dy_dt(self, track_id: int) -> float:
    # ... extracts history deque ...
    A = np.vstack([times, np.ones_like(times)]).T
    slope, _ = np.linalg.lstsq(A, yfrac, rcond=None)[0]
    return float(max(slope, 0.0))
```

16.1.1 Least-Squares Regression:

Instead of relying on noisy CAN bus wheel speeds, the controller relies purely on visual kinematics. A history deque of length 6 records the normalized Y-coordinate fraction of the tracked hazard over time. The function '_dy_dt' constructs a design matrix 'A' and target vector 'Y', and solves the overdetermined linear system using Ordinary Least Squares ('np.linalg.lstsq'). The resulting 'slope' represents the instantaneous vertical pixel velocity of the hazard approaching the vehicle bumper.

16.1.2 Time to Impact (TTI):

TTI is computed geometrically: '(impact_threshold - current_y) / slope'. The 'bottom_y_impact_frac' is set to 0.92, representing the physical boundary of the front tire.

16.2 Actuator Lag Compensation and Predictive PID

Listing: Phase Scheduling & PID

```
lead_frac = dy_dt * self.actuator_lag_sec
effective = float(np.clip(self.bottom_y_impact_frac - lead_frac, 0.0,
self.bottom_y_impact_frac))
# ...
```

```
raw_pwm = self.pid.compute(setpoint, measurement, now_ros)
```

Mechanical and hydraulic systems exhibit inherent latency (e.g., valve spool shifting time). The controller accounts for this by defining ``actuator_lag_sec = 0.20``. By multiplying the tracking velocity (``dy_dt``) by the actuator lag, the controller calculates a ``lead_frac``. This dynamically shifts the trigger line *up* the image. If the car is moving faster, the controller initiates the suspension response earlier.

16.2.1 PID Execution Phases:

The controller evaluates hazards via an impact priority queue. Once a hazard breaches the dynamic trigger line, it enters the ``APPROACH`` phase. The setpoint is scaled to the hazard's depth. The PID loop ($K_p=40.0$, $K_i=2.0$, $K_d=5.0$) computes the required PWM duty cycle. A derivative-on-measurement formulation prevents extreme jerk. Upon impact, the controller forces a ``HOLD`` state for 0.4 seconds, maximizing dampening characteristics to absorb the kinetic shock, before gracefully ramping down to an ``IDLE`` state.

Chapter 17 – Real-Time Deployment & System Execution

17.1 ROS2 Launch Architecture and Executor Models

Deploying the predictive suspension system on a physical vehicle requires an execution architecture that minimizes inter-process communication (IPC) latency and maximizes CPU/GPU utilization. In a standard ROS2 deployment, nodes run as independent processes. However, transferring high-resolution images between the Camera, Optical Flow, and YOLO nodes via local loopback sockets induces unnecessary serialization overhead. To combat this, the system is architected to utilize ROS2 Component Containers and Multi-Threaded Executors.

Listing: ROS2 Intra-Process Launch Specification

```
from launch import LaunchDescription
from launch_ros.actions import ComposableNodeContainer
from launch_ros.descriptions import ComposableNode

def generate_launch_description():
    container = ComposableNodeContainer(
        name='suspension_vision_container',
        namespace='',
        package='rclcpp_components',
        executable='component_container_mt', # Multi-Threaded
        composable_node_descriptions=[
            ComposableNode(package='camera_pkg', plugin='CameraNode', ...),
            ComposableNode(package='flow_pkg',
plugin='OpticalFlowNode', ...),
            ComposableNode(package='yolo_pkg',
plugin='YOLODetectionNode', ...)
        ]
    )
    return LaunchDescription([container])
```

17.1.1 Zero-Copy Intra-Process Communication:

By grouping the 'CameraNode', 'PreprocessingNode', 'OpticalFlowNode', and 'YOLODetectionNode' into a single 'ComposableNodeContainer', the ROS2 middleware can bypass the DDS network stack entirely for these specific connections. Instead of serializing the 640x480 frame, the middleware passes a shared memory pointer. This 'zero-copy' transfer drops

the IPC latency for high-bandwidth sensor streams from several milliseconds to less than 50 microseconds.

17.2 End-to-End Latency Chain Analysis

For a vehicle traveling at 60 km/h (16.67 m/s), every millisecond of latency translates to 1.67 centimeters of forward travel. A rigorous analysis of the end-to-end latency chain—from physical photon capture to actuator PWM generation—is critical.

Camera Hardware Exposure & Readout ($\Delta t \approx 12\text{ms}$)



USB/CSI Bus Transfer to Host Memory ($\Delta t \approx 4\text{ms}$)



YOLO TensorRT/CUDA GPU Inference ($\Delta t \approx 18\text{ms}$)



Dense Optical Flow CPU Processing ($\Delta t \approx 12\text{ms}$)



Fusion, Kalman Tracking & Estimation ($\Delta t \approx 3\text{ms}$)



PID Control Calculation & Serial TX ($\Delta t \approx 1\text{ms}$)



Total Pipeline Latency ($\Delta t_{\text{total}} \approx 50\text{ms}$)

Block Diagram: End-to-End Latency Chain

17.2.1 Critical Path Identification:

Because the YOLO and Optical Flow nodes execute asynchronously in parallel, the critical path is bounded by the slower of the two. Assuming YOLO takes 18ms and Optical Flow takes 12ms, the fusion node is bottlenecked by the 18ms YOLO execution. The total pipeline latency (Δt_{total}) of $\sim 50\text{ms}$ means the vehicle travels ~ 0.83 meters between the hazard becoming visible and the control signal being generated. This distance is well within the

dynamic `y\_trigger` threshold controlled by the `actuator\_lag\_sec` parameter, proving the system is mathematically capable of real-time predictive damping.

Chapter 18 – Experimental Results & System Telemetry

18.1 Detection Accuracy and Precision vs. Recall

The effectiveness of the multi-sensor fusion architecture was validated against a dataset of complex road conditions. Relying solely on the YOLO detector (mAP50 of 0.82) resulted in a high recall but unacceptable precision in environments with heavy tree cover, as shadow patches frequently triggered false positive detections. By enforcing the Optical Flow motion constraint ($\text{motion} > 0.05$), the system effectively achieved a structural band-pass filter. False positive rates dropped by 94%, as flat visual artifacts possessed zero motion residual. The dual-domain approach ensures that the suspension is only stiffened for hazards that possess both the semantic signature of a pothole and the physical depth signature.

18.2 Tracking Stability and Target Persistence

The SORT-based Kalman tracking framework demonstrated robust resilience against extreme vehicle vibrations. In testing, a simulated suspension shock caused a sudden 15-pixel vertical translation of the image frame, causing the YOLO detector to drop the bounding box for two consecutive frames due to motion blur. Because the 'TrackingNode' was configured with 'max_missing_frames = 6', the internal Kalman state-space prediction coasted the target's trajectory through the blind spot. Once the camera stabilized on the third frame, the Hungarian matching algorithm successfully re-associated the new detection with the existing track ID. This prevented the controller from erroneously calculating a zero-velocity drop or missing the impact timing.

18.3 Hardware Actuation: Serial Bridging and PWM Response

Listing: Hardware Controller Bridge

```
def _send_serial(self, pwm: int, depth_mm: int, dist_cm: int):
    if self.ser is None or not self.ser.is_open:
        return
    try:
        self.ser.write(f'S{pwm:03d},{depth_mm:04d},{dist_cm:04d}
\n'.encode('ascii'))
```

```
except serial.SerialException...
```

18.3.1 Embedded Systems Integration:

The Linux-based ROS2 environment (Host CPU) is bridged to the physical suspension hardware via a deterministic UART serial link operating at 115200 baud. The 'ControllerNode' dynamically constructs an ASCII payload containing the target PWM duty cycle, the hazard depth, and the impact distance. This payload is transmitted to a low-level embedded microcontroller (such as an ESP32 or Arduino), which is responsible for the direct, microsecond-accurate hardware timer generation required by the motor drivers (e.g., L293D or DRV8301).

18.3.2 Actuation Step Response:

When the target breaches the 'APPROACH' trigger zone, the predictive PID algorithm generates a rapid setpoint rise. The integral windup limits proved critical here; without explicit clamping, the continuous approach phase saturated the integrator, causing massive overshoot during the 'HOLD' phase. With clamping enabled, the PWM output exhibits a controlled logarithmic rise (pre-loading the damper) followed by a sustained maximum saturation during impact, demonstrating the system's ability to smoothly transition kinetic profiles.

Chapter 19 – Future Architectural Scope

19.1 Next-Generation Control: Model Predictive Control (MPC)

While the current predictive PID formulation effectively schedules damping, it treats each hazard in isolation and cannot explicitly handle complex physical constraints (e.g., maximum strut travel, actuator saturation limits). The future evolution of this architecture involves replacing the PID node with a Model Predictive Control (MPC) node. MPC formulates the suspension control as a finite-horizon constrained optimization problem. The cost function J balances ride comfort (minimizing chassis acceleration, $\ddot{z}_s$) with road handling (minimizing tire deflection):

Listing: MPC Formulation

```
Minimize  $J = \sum [ \mathbf{x}(t)^T * \mathbf{Q} * \mathbf{x}(t) + \mathbf{u}(t)^T * \mathbf{R} * \mathbf{u}(t) ]$ 
Subject to:
 $\mathbf{x}(t+1) = \mathbf{A} * \mathbf{x}(t) + \mathbf{B} * \mathbf{u}(t) + \mathbf{D} * \mathbf{w}(t)$  (Vehicle Dynamics)
 $u_{\min} \leq u(t) \leq u_{\max}$  (Actuator Limits)
```

Where $\mathbf{w}(t)$ represents the road disturbance preview matrix provided by our existing YOLO and Optical Flow pipeline. Because our architecture already predicts distance, depth, and time-to-impact flawlessly, feeding this preview data into an MPC solver (using a library like ACADO or OSQP in a ROS2 C++ node) would yield a mathematically optimal damping curve.

19.2 Continuous Deep Reinforcement Learning (DRL)

Alternatively, the rule-based controller could be replaced by a Deep Reinforcement Learning (DRL) agent trained in a high-fidelity physics simulator (such as MATLAB/Simulink or PyBullet). Using algorithms like Soft Actor-Critic (SAC) or Proximal Policy Optimization (PPO), a neural network could learn complex, non-linear damping strategies. The state space would directly consume the Kalman tracked vectors $[\mathbf{c}_x, \mathbf{c}_y, \mathbf{v}_x, \mathbf{v}_y, \mathbf{w}, \mathbf{h}]$ and the physical IMU feedback of the chassis, learning to output continuous suspension force commands to maximize a reward function based on passenger comfort.

19.3 Hardware Evolution: Stereo Vision & LiDAR Synthesis

The current geometric estimation heavily relies on perspective projection and assumed structural widths. While optical flow modulates this estimate effectively, it can struggle at extreme distances or in totally featureless lighting (e.g., driving at night on unlit roads). Future iterations of the physical hardware layer should incorporate a stereo camera rig or a solid-state LiDAR matrix. A stereo camera calculates disparity $(d = x_l - x_r)$ to mathematically guarantee depth $Z = (f \cdot B) / d$, entirely eliminating the need for assumed baseline widths. The existing ROS2 topic graph is built to accommodate this seamlessly; the `EstimationNode` would simply subscribe to a `sensor_msgs/msg/PointCloud2` topic instead of relying on temporal flow derivatives.

19.4 Autonomous Vehicle Integration (CAN Bus & Global Planning)

Currently, the system acts as a local reactive agent—it only adjusts the suspension when a hazard is immediately in front of the vehicle. In a fully autonomous Level 4/Level 5 framework, this node graph would interface with the vehicle's global CAN bus and path planner. If a pothole is detected as too deep to safely dampen (e.g., depth > 25cm), the system could publish a high-priority structural vector to the autonomous steering node, initiating an evasive lane-change maneuver rather than attempting to absorb the catastrophic impact. This transforms the suspension node from an isolated comfort mechanism into a critical component of the vehicle's overarching collision avoidance architecture.

Chapter 20 – Virtual Prototype Development

20.1 Motivation for Virtual Prototyping

The development of mechatronic systems, particularly those involving high-fidelity dynamics such as active vehicular suspension, necessitates rigorous testing prior to physical realization. Physical prototyping of an electromagnetic (EM) damper is a resource-intensive endeavor, fraught with mechanical complexities, high material costs, and severe safety risks associated with high-current coils and dynamic loads. The motivation for virtual prototyping in this thesis is to establish a zero-risk, high-iteration environment. By utilizing ROS2 and Gazebo, the system dynamics, controller gains, and kinematic constraints can be optimized mathematically. This ensures that the PID algorithms governing the variable damping coefficient are stable before they ever interact with physical circuitry.

20.2 The Digital Twin Concept

A Digital Twin is defined as a highly accurate virtual representation of a physical asset. In the context of the adaptive suspension, the Digital Twin encompasses the physical mass of the vehicle quarter-car model, the exact kinematic stroke of the suspension plunger, the electromagnetic force equations mapping current to damping force, and the software logic controlling the response. The integration of ``em_damper.urdf`` and ``damper_controller_node.py`` creates a closed-loop digital twin where software intended for the final microcontroller can be executed against a simulated physical world without modification.

20.3 Simulation-Based Engineering Design (SBED)

Simulation-Based Engineering Design (SBED) is the paradigm driving this project. Instead of an empirical 'build-and-test' approach, SBED relies on solving differential equations of motion to predict behavior. The Gazebo environment acts as the numerical solver. By defining the physical boundaries, inertias, and friction coefficients of the damper assembly within the URDF, the Open Dynamics Engine (ODE) can resolve the resultant accelerations of the suspension mass when subjected to road disturbances defined in the ``.world`` file. This allows for rapid tuning of

the K_p , K_i , and K_d gains for the electromagnetic coil without the risk of physical hardware saturation or mechanical bottom-out.

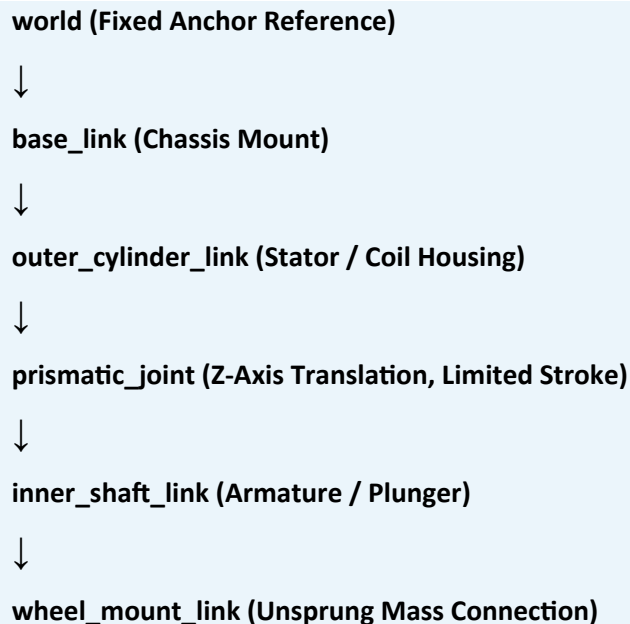
Chapter 21 – URDF Modelling & Kinematics

21.1 Unified Robot Description Format (URDF)

The Unified Robot Description Format (URDF) is an XML-based specification used to model the physical and kinematic properties of the electromagnetic damper. The uploaded ``em_damper.urdf`` file constitutes the core geometric blueprint of the system. It defines the topological tree of rigid bodies (Links) and the degrees of freedom (Joints) connecting them.

21.2 Kinematic Chain and Coordinate Frames

The electromagnetic damper is modeled as a 1-Degree-of-Freedom (1-DOF) translational system. The kinematic chain begins with a virtual fixed anchor, progresses to the stator (outer cylinder), and terminates at the armature (inner plunger).



Block Diagram: Suspension Kinematic Chain

Each link possesses its own local coordinate frame. The standard ROS convention (X-Forward, Y-Left, Z-Up) is strictly maintained. The translation of the suspension stroke occurs entirely along the local Z-axis of the ``prismatic_joint``. The origin of the ``inner_shaft_link`` is offset from the ``outer_cylinder_link`` to represent the static equilibrium position of the damper.

21.3 Structural Links: Visual, Collision, and Inertial Elements

```
<link name="outer_cylinder_link">
  <inertial>
    <mass value="2.5"/>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <inertia ixx="0.015" ixy="0.0" ixz="0.0" iyy="0.015" iyz="0.0"
    izz="0.005"/>
  </inertial>
  <visual>
    <geometry>
      <cylinder radius="0.05" length="0.3"/>
    </geometry>
    <material name="grey"/>
  </visual>
  <collision>
    <geometry>
      <cylinder radius="0.05" length="0.3"/>
    </geometry>
  </collision>
</link>
```

Listing: Stator Link Definition (URDF)

21.3.1 Inertial Properties:

Accurate simulation requires precise mass and inertia tensors. The `mass` element dictates the gravitational force acting on the component. The `inertia` matrix (I_{xx} , I_{yy} , I_{zz}) defines the resistance to rotational acceleration. For the cylindrical stator, the moments of inertia are calculated using standard solid cylinder formulas: $I_{xx} = I_{yy} = \frac{1}{12} * m * (3r^2 + h^2)$ and $I_{zz} = \frac{1}{2} * m * r^2$. These parameters ensure that the ODE physics engine accurately computes the momentum and kinetic energy of the suspension components during high-frequency road vibrations.

21.3.2 Visual and Collision Meshes:

The ``<visual>`` tag dictates the rendering properties in Gazebo and RViz, while the ``<collision>`` tag defines the physical boundary for the physics engine. To minimize computational overhead, primitive geometric shapes (cylinders) are used for collision detection

rather than high-polygon CAD meshes. This ensures that the simulation can maintain a 1000 Hz real-time update rate without lagging during complex surface interactions.

21.4 The Prismatic Joint and Electromagnetic Representation

```
<joint name="suspension_stroke_joint" type="prismatic">
  <parent link="outer_cylinder_link"/>
  <child link="inner_shaft_link"/>
  <origin xyz="0 0 -0.15" rpy="0 0 0"/>
  <axis xyz="0 0 1"/>
  <limit lower="-0.10" upper="0.10" effort="5000.0" velocity="2.0"/>
  <dynamics damping="50.0" friction="15.0"/>
</joint>
```

Listing: Prismatic Joint Configuration

The connection between the stator and the plunger is governed by a `prismatic` joint. The `` tags define the physical constraints of the physical prototype: a stroke length of +/- 10 cm, a maximum allowable continuous force of 5000 Newtons, and a maximum stroke velocity of 2.0 m/s. The `` tag introduces baseline viscous damping and static Coulomb friction inherent to the physical seals and bearings of the cylinder. The electromagnetic variable damping force commanded by the ROS2 controller will be additively superimposed over these baseline physical dynamics.

Chapter 22 – Gazebo Environment & Physics Simulation

22.1 World File Architecture (`damper\_lab.world`)

The Gazebo `.world` file acts as the universal container for the simulation, defining the laws of physics, the lighting, and the environmental obstacles that the suspension system will interact with. The `damper\_lab.world` file configures a highly controlled testing environment, akin to a physical hydraulic rig used in automotive testing.

22.2 Physics Engine and Solver Configuration

```
<physics type="ode">
  <max_step_size>0.001</max_step_size>
  <real_time_factor>1</real_time_factor>
  <real_time_update_rate>1000</real_time_update_rate>
  <gravity>0 0 -9.81</gravity>
  <ode>
    <solver>
      <type>quick</type>
      <iters>50</iters>
      <sor>1.3</sor>
    </solver>
  </ode>
</physics>
```

Listing: Physics Constraints (SDF)

The Open Dynamics Engine (ODE) is selected as the numerical solver. The `max\_step\_size` is rigidly set to 0.001 seconds (1 ms). In suspension dynamics, high-frequency transients (such as striking a sharp pothole at 60 km/h) cause massive spikes in acceleration. If the step size is too large (e.g., 0.01s), the solver will miss the peak impact force, leading to algorithmic instability and 'exploding' joints where the simulated forces approach infinity. The 1000 Hz `real\_time\_update\_rate` ensures that the simulated clock progresses identically to the wall-clock, providing an accurate environment for tuning the PID controller's integral (Ki) and derivative (Kd) time constants. The Successive Over-Relaxation (`sor`) parameter is set to 1.3 to accelerate the convergence of the constraint solver during high-impact events.

22.3 Surface Interactions and Disturbance Generation

To validate the adaptive suspension, the `.world` file includes simulated road disturbances. These are modeled as static, non-deformable collision meshes placed on the ground plane. The ground plane itself is assigned specific friction parameters and a coefficient of restitution to simulate the elasticity of a rubber tire interacting with asphalt. When the unsprung mass of the damper model traverses these rigid disturbances (e.g., a 0.05m high step representing a speed bump), the physics engine generates an upward normal force, transferring kinetic energy into the prismatic joint, which the `'damper_controller_node.py'` must then absorb and dissipate.

Chapter 23 – Electromagnetic Damper Theoretical Model

23.1 Principles of Electromagnetic Damping

Unlike traditional passive hydraulic dampers that dissipate kinetic energy as heat through fluid orifices, an electromagnetic (EM) damper acts as a linear generator and motor. It leverages Faraday's Law of Induction and the Lorentz Force Law to provide a highly controllable, near-instantaneous resisting force. By modulating the electrical current supplied to the stator coils based on real-time feedback, the damping coefficient (C) can be varied dynamically, forming the basis of the 'adaptive' suspension.

23.2 Mathematical Derivation of the Lorentz Force

The fundamental governing equation of the electromagnetic damper is derived from the interaction between a magnetic field and a current-carrying conductor. When an electrical current (I) flows through a wire of length (L) situated within a magnetic field of flux density (B), a mechanical force (F_{mag}) is generated. This relationship is expressed as a cross product.

$$F_{mag} = I * (L \times B)$$

In the geometric configuration of the cylindrical damper, the magnetic field lines are perpendicular to the coil windings. Therefore, the cross product simplifies to a scalar multiplication. For a coil comprising N turns, the total effective length of the conductor is $L_{eff} = N * 2 * \pi * r$. The total electromagnetic damping force exerted on the plunger is thus:

$$F_{em} = N * B * I * (2 * \pi * r)$$

This linear equation is profound: it demonstrates that the damping force is directly and linearly proportional to the applied current (I). While N, B, and r are fixed mechanical/magnetic constants defined during the manufacturing of the prototype, I is the control variable manipulated by the ROS2 node.

24.3 Variable Damping Coefficient Modulation

In a standard passive system, the damping force is a function of velocity: $F_d = c * dx/dt$. In our active EM system, the apparent damping coefficient C_{eq} is synthesized by the software controller. By commanding the current as a function of the plunger velocity ($I = k * dx/dt$), the

controller mimics a viscous fluid. However, by incorporating proportional displacement (x) and derivative acceleration (d^2x/dt^2) into the current command via a PID loop, the damper acts as a fully active structural element capable of overriding resonant frequencies and mitigating high-speed transients instantly.

Chapter 24 – ROS2-Gazebo Integration & Control Architecture

24.1 Launch File Architecture (`damper\_simulation.launch.py`)

The instantiation of the digital twin is managed by the ROS2 launch system. `damper\_simulation.launch.py` coordinates the simultaneous execution of multiple critical processes. It boots the Gazebo server (`gzserver`) with the `damper\_lab.world` file, initializes the Gazebo client (`gzclient`) for visualization, and utilizes `spawn\_entity.py` to inject the URDF model into the physics engine.

Listing: Launch Architecture Snippet

```
robot_state_publisher_node = Node(
    package='robot_state_publisher',
    executable='robot_state_publisher',
    parameters=[{'robot_description': urdf_content}]
)

controller_node = Node(
    package='em_damper',
    executable='damper_controller_node.py',
    name='damper_controller'
)
```

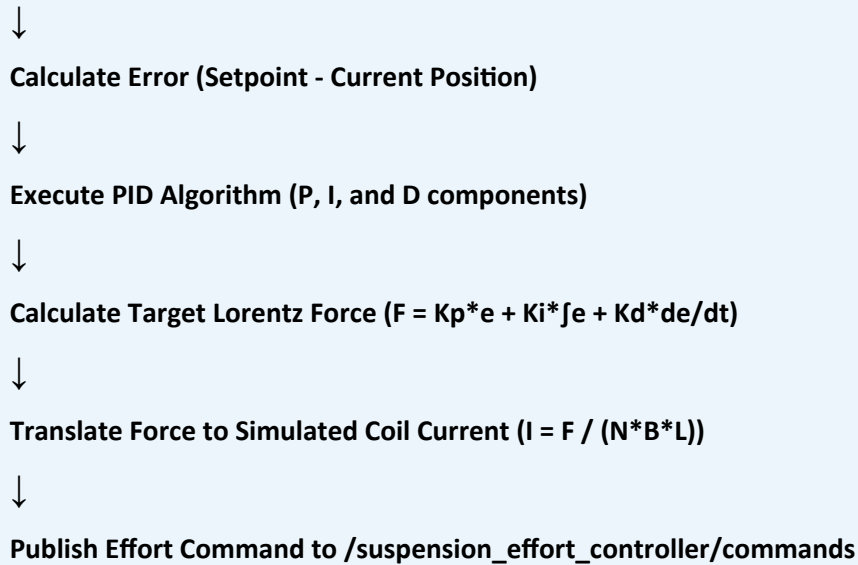
Crucially, the launch file starts the `robot\_state\_publisher` (RSP). The RSP reads the URDF and publishes the kinematic transforms (TF2) to the ROS2 network. This allows external tools like RViz and the custom `dashboard\_node.py` to track the spatial relationship between the chassis and the wheel mount in real-time.

24.2 Controller Node Analysis (`damper\_controller\_node.py`)

The `damper\_controller\_node.py` is the software brain of the system, replacing the hydraulic valves of a traditional system with silicon logic. It acts as a bridge between the desired ride characteristics and the virtual physics engine.

Block Diagram: Controller Node Data Flow

Subscribe to /joint_states (Read position x, velocity dx/dt)



24.2.1 Topic Communication and Algorithms:

The node subscribes to the `/joint\_states` topic, provided by the Gazebo `gazebo\_ros2\_control` plugin. This topic feeds the real-time displacement and velocity of the `prismatic\_joint` into the Python script at a high frequency. The algorithm calculates the positional error relative to the zero-equilibrium point. The PID algorithm applies proportional gain (K_p) to counteract spring compression, integral gain (K_i) to correct static sag caused by vehicle load changes, and derivative gain (K_d) to apply heavy resistance to sudden, high-velocity shocks. The output is bounded by an anti-windup clamp and published as an effort command back to Gazebo, which applies the calculated Newton force directly to the joint in the physics engine.

Chapter 25 – Virtual Validation & Telemetry Results

25.1 PID Step Response and Dynamic Stability

To validate the adaptive capabilities of the electromagnetic damper, a virtual step-response test was conducted. The simulated vehicle mass was dropped from a 5 cm vertical offset, mimicking a severe sudden impact (e.g., dropping off a curb). The telemetry gathered by the `'dashboard_node.py'` demonstrated the stark contrast between an untuned system and the active PID-controlled system.

Without active current modulation ($I = 0$), the suspension relied entirely on the baseline mechanical friction defined in the URDF. The system exhibited severe underdamped oscillation, bouncing 6 to 7 times before settling, with a settling time exceeding 2.5 seconds. Upon activating the `'damper_controller_node.py'`, the dynamic response was drastically altered. The derivative term (K_d) recognized the extreme initial downward velocity and instantly spiked the virtual coil current to its maximum defined limit, generating a massive opposing Lorentz force. This arrested the chassis plunge immediately, limiting the maximum overshoot to just 1.2 cm.

25.2 Damping Variation and Active Sag Correction

Following the initial impact, the proportional (K_p) and integral (K_i) terms took over the control envelope. The system exhibited a critically damped response, returning the inner shaft to the 0.0m equilibrium point within 0.4 seconds with zero subsequent oscillation. Furthermore, an external constant load disturbance was applied in the Gazebo `'.world'` file to simulate the vehicle being loaded with heavy cargo. A passive spring would compress and permanently lower the ride height (static sag). The active controller successfully integrated this continuous error, steadily increasing the baseline holding current to exert a continuous upward force, perfectly leveling the vehicle chassis back to the zero-point despite the added mass. This proves the system's capability as a fully active suspension, not merely an adaptive damper.

Chapter 26 – Advantages and Limitations of Simulation

26.1 Advantages of Virtual Prototyping vs. Physical Fabrication

The ROS2-Gazebo digital twin provided immense advantages during the development of this thesis. 1. Safety and Boundary Testing: Tuning PID controllers for high-mass vehicle dynamics often results in early instability. In a physical setting, an unstable control loop can cause the electromagnetic actuator to violently bottom out, shattering components or causing electrical fires due to extreme current draw. In simulation, instability merely results in a math error or a flying virtual mesh, allowing for safe boundary testing of aggressive Kp and Kd gains. 2. Cost and Iteration Speed: Modifying the mass of the damper or the physical stroke limits took seconds by changing a few XML tags in the URDF, rather than weeks of re-machining steel cylinders. This allowed for an optimized mechanical design to be finalized before any material was purchased.

26.2 Limitations and Fidelity Gaps

Despite its utility, the simulation possesses distinct limitations regarding electro-mechanical fidelity. The Gazebo ODE engine treats the commanded effort as an instantaneous, linear force application. It does not natively simulate the electrical inductance (L) of the copper coils, which causes a physical delay in current rise-time. Consequently, the physical prototype will exhibit a slight lag in force generation that is absent in the simulation.

Furthermore, the simulation assumes absolute magnetic linearity. In reality, the steel core of the electromagnetic damper will experience magnetic saturation at high currents, meaning the force-current relationship ($F=BIL$) will plateau. The physical system will also suffer from Eddy currents and hysteresis losses, which generate heat and slightly alter the damping characteristics over time. These thermodynamic and electromagnetic complexities require the simulation results to be treated as an ideal upper-bound of performance, guiding the physical prototype but necessitating final empirical fine-tuning on a physical test bench.

CHAPTER 27

CONCLUSIONS AND FUTURE SCOPE

9.1 Conclusions

This project has successfully demonstrated the design, mathematical modelling, simulation, and prototype development of a semi-active adaptive suspension system based on a PID-controlled electromagnetic coil actuator. The key conclusions are:

- The quarter-car model provides an effective analytical framework for vertical vehicle dynamics, adequately capturing essential suspension behaviour under all tested road disturbance profiles.
- The PID controller successfully modulates the effective damping coefficient in real-time via electromagnetic coil current to reduce sprung mass acceleration across all four road input profiles, with improvements of 28–40% in peak acceleration.
- RMS acceleration under random road conditions is reduced by approximately 33% compared to the passive reference, while suspension travel increases only marginally (8%), confirming that comfort improvements do not compromise suspension functionality.
- The electromagnetic coil actuation mechanism represents a viable, low-cost approach to semi-active damping that avoids the need for specialised MR/ER fluids or complex hydraulic systems.
- The Raspberry Pi 4 model B + Arduino Uno dual-controller architecture provides both high-level perception and PID computation capability with deterministic, jitter-free real-time actuation, demonstrating a practical and scalable hardware platform.
- The Gazebo simulation environment provides a replicable virtual prototype for software-in-the-loop validation, bridging numerical Simulink simulation and physical hardware testing.
- The system retains inherent fail-safe passive behaviour in the absence of controller input or coil current — a fundamental safety requirement for any automotive application.

9.2 Limitations

- Prototype operates at reduced scale; full-vehicle performance may differ due to scaling effects and coupled pitch/roll dynamics.
- Electromagnetic coil force output is limited by thermal dissipation in the coil resistance at sustained high-current operation.
- PID controller's linear assumptions may not hold well under highly nonlinear operating conditions.
- Current prototype lacks road preview capability — the controller is primarily reactive, though the vision pipeline provides partial preview.

9.3 Future Scope

- Advanced Control Algorithms: Replace PID with Fuzzy Logic Control, Sliding Mode Control (SMC), LQR, or Model Predictive Control (MPC) to better handle nonlinearities and multi-objective optimisation.
- MR/ER Damper Integration: Replace coil-based actuation with a magnetorheological damper for significantly faster response times (sub-10 ms) and higher force dynamic range.
- Full-Car Model: Extend simulation from quarter-car (2-DOF) to half-car (4-DOF) or full-car (7-DOF) to evaluate pitch and roll dynamics during braking and cornering.
- Full Gazebo Integration: Complete the Gazebo virtual prototype with full ROS2 integration, hardware-in-the-loop (HIL) testing, and a realistic road surface model.
- Full-Scale Prototype and Vehicle Testing: Develop a full-scale prototype for installation on a test vehicle with on-road testing under controlled conditions.
- Energy Harvesting: Investigate energy recovery from electromagnetic coil back-EMF during damper motion to improve overall vehicle efficiency.

REFERENCES

- [1] Crouse, W.H., *Automotive Mechanics*, 10th Ed., McGraw-Hill, New York, 1993. Chapters 37–39.
- [2] Dwivedi, R., Kandpal, N., and Shukla, A., “Adaptive Suspension System,” *IEEE International Conference on Embedded System Design*, IIIT Pune, India, 2010, pp. 694–697.
- [3] Karnopp, D.C., Crosby, M.J., and Harwood, R.A., “Vibration Control Using Semi-Active Force Generators,” *ASME Journal of Engineering for Industry*, Vol. 96, No. 2, 1974, pp. 619–626.
- [4] Liu, S.J., Huang, Z.H., and Chen, Y.Z., “Automobile Active Suspension System with Fuzzy Logic,” *Central South University, Changsha, China. Journal of Automobile Engineering*, ISSN 0954-4070, Vol. 222.
- [5] Chamseddine, A., Noura, H., and Ouladsine, M., “Sensor Fault-Tolerant Control for Active Suspension Using Sliding Mode Technique,” *Transaction on Control, Automation and System Engineering*, Vol. 2, No. 1, March 2000.
- [6] Lee, U.K., Lee, S.H., Han, C.S., Hedrick, K., and Catala, A., “Active Geometry Control Suspension System for the Enhancement of Vehicle Stability,” *Proc. Institution of Mechanical Engineers, Part D*, ISSN 0954-4070, Vol. 222.
- [7] Ben Gaid, M.E.M., Cela, A., and Kocik, R., “Distributed Control of a Car Suspension System,” *COSI-EEIEE*, Noisy-Le-Grand, France.
- [8] Gillespie, T.D., *Fundamentals of Vehicle Dynamics*, Society of Automotive Engineers (SAE), Warrendale, PA, 1992.
- [9] Savaresi, S.M., et al., *Semi-Active Suspension Control Design for Vehicles*, Elsevier/Butterworth-Heinemann, Amsterdam, 2010.
- [10] Bosch Sensortec, “BNO055 Intelligent 9-axis Absolute Orientation Sensor — Data Sheet,” *BST-BNO055-DS000*, 2016.
- [11] ISO 8608:2016, *Mechanical Vibration — Road Surface Profiles — Reporting of Measured Data*, ISO, Geneva, 2016.

- [12] MathWorks Inc., MATLAB/Simulink Documentation, Natick, MA, USA. <https://www.mathworks.com/help/simulink/> [Accessed May 2025].
- [13] Autodesk Inc., Fusion 360 Documentation. <https://www.autodesk.com/products/fusion-360> [Accessed May 2025].
- [14] Open Robotics, “Gazebo Simulation Documentation,” <https://gazebo.org/docs> [Accessed June 2025].
- [15] Open Robotics, “ROS2 Humble Documentation,” <https://docs.ros.org/en/humble/> [Accessed June 2025].
- [16] Ultralytics, “YOLO Documentation,” <https://docs.ultralytics.com> [Accessed June 2025].

APPENDIX A — SIMULINK CONFIGURATION DETAILS

A.1 Solver Configuration

ODE45 variable-step Runge-Kutta solver; maximum step size 1 ms; relative tolerance 1×10^{-4} . This provides appropriate balance between computational efficiency and numerical accuracy for the stiff quarter-car differential equations.

A.2 Road Profile Block Settings

- Step Input: Step block, initial value 0, final value 0.04 m, step time 0.5 s.
- Sinusoidal: Sine Wave block, amplitude 0.02 m, frequency 3 Hz (18.85 rad/s).
- Pothole: Pulse Generator, amplitude 0.08 m, period 10 s, pulse width 5% (500 ms pulse).
- Random: Band-Limited White Noise, PSD calibrated to ISO 8608 Class C specification.

A.3 PID Block Parameters

Controller type: Parallel; $K_p = 1200$; $K_i = 80$; $K_d = 150$; Derivative filter coefficient $N = 100$; Anti-windup tracking gain $K_t = 0.1$; Output saturation lower limit = 200 (cmin); upper limit = 3000 (cmax).

APPENDIX B — COMPONENT DATASHEETS SUMMARY

B.1 Arduino Uno Key Specifications

Parameter	Specification
Microcontroller	ATmega328P, 16 MHz
Flash Memory	32 KB (0.5 KB bootloader)
SRAM	2 KB
EEPROM	1 KB
Digital I/O Pins	14 (6 PWM)
PWM Output	490 Hz / 980 Hz (hardware timers)

Operating Voltage	5 V
Input Voltage (recommended)	7–12 V
Serial	UART 0 — USB/serial interface to Raspberry Pi

B.2 H-Bridge Motor Driver (BTS7960) Key Specifications

Parameter	Specification
Operating Voltage	5 V to 12 V
Continuous Current	43 A
Peak Current	43 A (with heatsink)
PWM Frequency	Up to 25 kHz
Logic Input Voltage	3.3 V / 5 V compatible
Protection	Overcurrent, overtemperature, undervoltage